Chinese Multi-Cinema (CMC) Ticketing System

Software Requirements Specification

Version 4.0

12 November 2026

Group #14:

Luke F., Matthew W., Joshua H.

Prepared for

CS 250 - Introduction to Software Systems

Instructor: Gus Hanna, Ph.D.

Fall 2026

Revision History

Date	Description	Author	Comments
10 Sep 2026	Version 0.1 - Title page and outline	Luke F., Matthew W.	Filled out title page with project name, date, group number, and names.
17 Sep 2026	Version 0.2 - Requirements draft	Joshua H., Luke F., Matthew W.	Drafted Sections 1.1-3.1 and 3.5.
24 Sep 2026	Version 1.0 - Complete SRS (Sections 1-3.5)	Joshua H., Luke F., Matthew W.	Completed Sections 1-3.5: functional requirements, use-case diagram and descriptions, classes, and non-functional requirements.
8 Oct 2026	Version 2.0 - Software Design Specification	Joshua H., Luke F., Matthew W.	Added Section 4: system description, software architecture, UML class diagram, class descriptions, and development plan.
22 Oct 2026	Version 3.0 - Test Plan	Joshua H., Luke F., Matthew W.	Added Section 4.6 (design updates) and Section 5: test strategy, test-scope diagram, unit, integration, functional, and system test sets, ten sample test cases, traceability, and test schedule.
12 Nov 2026	Version 4.0 - Design 2.0: Architecture and Data Management	Joshua H., Luke F., Matthew W.	Added Section 6: updated architecture diagram with all data stores (Figure 5), design changes since Version 2.0, data management strategy with logical data model (Figure 6), and discussion of alternatives and tradeoffs. Added database terms to 1.3 and references to 1.4.

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Luke F.	Software Eng.	9/24/26
	Matthew W.	Software Eng.	9/24/26
	Joshua H.	Software Eng.	9/24/26
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History **2**

Document Approval **2**

1. Introduction **6**
1.1 Purpose . . . 6
1.2 Scope . . . 6
1.3 Definitions, Acronyms, and Abbreviations . . . 7
1.4 References . . . 7
1.5 Overview . . . 8

2. General Description **8**
2.1 Product Perspective . . . 8
2.2 Product Functions (User Requirements) . . . 9
2.3 User Characteristics . . . 9
2.4 General Constraints . . . 10
2.5 Assumptions and Dependencies . . . 10

3. Specific Requirements **10**
3.1 External Interface Requirements . . . 10
3.1.1 User Interfaces . . . 10
3.1.2 Hardware Interfaces . . . 11
3.1.3 Software Interfaces . . . 11
3.1.4 Communications Interfaces . . . 11
3.2 Functional Requirements . . . 11
3.2.1 Browse Movies and Showtimes . . . 11
3.2.2 Select and Hold Tickets . . . 12
3.2.3 Complete Purchase and Deliver Tickets . . . 13
3.2.4 Customer Accounts and Membership . . . 13
3.2.5 Box Office Sales, Ticket Validation, and Refunds . . . 14
3.2.6 Administration . . . 14
3.2.7 Reporting, Feedback, and Demand Protection . . . 15
3.3 Use Cases . . . 15
3.3.1 Use Case #1 (UC-01): Browse Movies and Showtimes . . . 16
3.3.2 Use Case #2 (UC-02): Purchase Tickets . . . 17
3.3.3 Use Case #3 (UC-03): Select Assigned Seats (extends UC-02) . . . 17
3.3.4 Use Case #4 (UC-04): Submit Satisfaction Rating (extends UC-02) . . . 18
3.3.5 Use Case #5 (UC-05): Manage Account and Membership . . . 18
3.3.6 Use Case #6 (UC-06): Sell Tickets at Box Office . . . 18
3.3.7 Use Case #7 (UC-07): Validate Ticket . . . 19
3.3.8 Use Case #8 (UC-08): Process Refund . . . 19
3.3.9 Use Case #9 (UC-09): Manage Movies, Showtimes and Pricing . . . 19
3.3.10 Use Case #10 (UC-10): Correct Transaction (Override) . . . 20
3.3.11 Use Case #11 (UC-11): View Sales Reports . . . 20
3.4 Classes / Objects . . . 20
Enumerated Types . . . 20
3.4.1 Theater . . . 21
3.4.2 Auditorium . . . 21
3.4.3 Movie . . . 22
3.4.4 Showtime . . . 22
3.4.5 TicketHold . . . 22
3.4.6 Ticket . . . 23
3.4.7 Transaction . . . 23
3.4.8 CustomerAccount . . . 24
3.4.9 EmployeeAccount . . . 24

3.4.10 AuditRecord . . . 25
3.5 Non-Functional Requirements . . . 25
3.5.1 Performance . . . 25
3.5.2 Reliability . . . 25
3.5.3 Availability . . . 26
3.5.4 Security . . . 26
3.5.5 Maintainability . . . 26
3.5.6 Portability . . . 27
3.5.7 Usability and Accessibility . . . 27

4. Software Design Specification **27**
4.1 System Description . . . 27
4.2 Software Architecture Overview . . . 27
4.3 UML Class Diagram . . . 29
4.4 Class Descriptions . . . 30
4.4.1 Theater . . . 30
4.4.2 Auditorium . . . 31
4.4.3 Movie . . . 31
4.4.4 Showtime . . . 31
4.4.5 TicketHold . . . 32
4.4.6 Ticket . . . 32
4.4.7 Transaction . . . 33
4.4.8 Account . . . 33
4.4.9 CustomerAccount . . . 33
4.4.10 EmployeeAccount . . . 34
4.4.11 AuditRecord . . . 34
4.4.12 PaymentGateway . . . 34
4.5 Development Plan and Timeline . . . 34
4.5.1 Partitioning of Tasks . . . 34
4.5.2 Team Member Responsibilities . . . 35
4.6 Design Updates Since Version 2.0 . . . 35

5. Test Plan **36**
5.1 Test Objectives . . . 36
5.2 Test Strategy . . . 36
5.2.1 Test Levels . . . 36
5.2.2 Test Design Techniques . . . 37
5.2.3 Test Environment and Test Data . . . 37
5.2.4 Entry and Exit Criteria . . . 38
5.3 Test Scope on the Design . . . 38
5.4 Unit Test Sets . . . 40
5.4.1 TS-U1: Showtime Sales Window and Seat Status . . . 40
5.4.2 TS-U2: Ticket Validation and Refund States . . . 40
5.4.3 TS-U3: TicketHold Expiry and Ticket Limits . . . 41
5.5 Integration Test Sets . . . 41
5.5.1 TS-I1: Purchase Pipeline . . . 41
5.5.2 TS-I2: Refund and Audit Path . . . 42
5.6 Functional Test Sets . . . 42
5.6.1 TS-F1: Purchase Tickets (UC-01, UC-02, UC-03) . . . 42
5.6.2 TS-F2: Validate Tickets and Process Refunds (UC-07, UC-08) . . . 43
5.6.3 TS-F3: Manage Showtimes and Prices (UC-09) . . . 44
5.7 System Test Sets . . . 44
5.7.1 TS-S1: Seat Integrity and Cross-Channel Consistency . . . 44
5.7.2 TS-S2: Performance, Load, and Availability . . . 44
5.7.3 TS-S3: Security . . . 45
5.7.4 TS-S4: Usability and Accessibility (Acceptance) . . . 45
5.8 Test Case Samples . . . 45

5.9 Requirements Traceability 46
5.10 Test Schedule and Responsibilities 47

6. Design 2.0: Architecture and Data Management **47**
6.1 Updated Software Architecture 47
6.2 Modifications from the Initial Design (Version 2.0) 50
6.3 Data Management Strategy 51
6.3.1 Overview 51
6.3.2 Logical Organization of the Data 51
6.3.3 Data Integrity and Consistency 53
6.3.4 Sensitive Data and Security 54
6.3.5 Backup, Recovery, and Retention 54
6.4 Design Decisions, Alternatives, and Tradeoffs 55
6.4.1 SQL or NoSQL 55
6.4.2 How Many Databases 55
6.4.3 Other Design Decisions 56

1. Introduction

Chinese Multi-Cinema (CMC) operates 20 movie theaters in the San Diego area. Its current ticketing software handles basic sales, but years of patches have made it slow and unreliable, and customers have no convenient way to buy tickets ahead of time. CMC needs a single, modern ticketing system shared by every theater, kiosk, and box office, so that customers can buy tickets in a few minutes from anywhere and every employee sees the same showtimes, prices, and seat availability.

The new CMC Ticketing System will let customers browse movies and showtimes, buy up to 20 tickets at a time, choose their seats in deluxe auditoriums, and receive tickets by email or as a printout, either as a guest or with an optional account that tracks loyalty points and membership. Theater employees will use the same system to sell, scan, and refund tickets, administrators will use it to manage movies, showtimes, and prices, and CMC management will use it to review sales across all locations. Sections 1-3 describe what the system must do and how well it must do it; Section 4 describes how it will be built; Section 5 describes how it will be tested; and Section 6 updates the design with how the system's data will be stored and protected.

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete set of requirements for the CMC Ticketing System. It gives everyone involved one agreed description of the system before design and implementation begin, so that the finished product can be checked against it.

The intended audience is:

- **CMC management and theater staff**, who confirm that the requirements reflect their needs (mainly Sections 1 and 2)
- **Software designers and developers**, who build the system from the detailed requirements in Section 3
- **Testers** who write test cases from the numbered requirements and use cases

1.2 Scope

The software product specified here is the **CMC Ticketing System**, a browser-based ticketing platform used by the public website, in-theater self-service kiosks, and box-office workstations at all 20 CMC theaters.

The system will:

- show movies, showtimes, prices, remaining availability, and licensed review scores for every CMC theater.
- sell up to 20 tickets per transaction, with assigned seating in 75-seat deluxe auditoriums and general admission in 150-seat regular auditoriums.
- hold selected tickets for five minutes during checkout so they cannot be sold twice.
- accept card and PayPal payments through a third-party processor and deliver tickets by email or print.
- support guest checkout and optional customer accounts with purchase history, loyalty points, and CMC membership.
- let employees sell, validate, and refund tickets, and let administrators manage movies, showtimes, auditoriums, prices, and transaction corrections.
- provide sales and revenue reports to CMC management.

The system will not (in this release) include a native mobile app, online self-service refunds, concession or food ordering, automatic nearby-theater recommendations, or the processing and storage of raw card data (handled by the payment processor).

Objectives. The system is successful when: 90% of first-time customers can complete a purchase without help in a median time of three minutes or less; no assigned seat is ever sold twice; a sale on any channel appears on every other channel within two seconds; and the customer site is available at least 99.9% of each month. Section 3.5 states these objectives as measurable requirements.

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability: the guarantees a database transaction gives that a group of changes is saved completely or not at all.
Administrator	An authorized CMC employee who manages movies, showtimes, auditoriums, prices, and transaction corrections.
API	Application Programming Interface; a defined way for two software systems to exchange data.
Auditorium (regular / deluxe)	A screening room. Regular auditoriums hold 150 general-admission seats; deluxe auditoriums hold 75 assigned seats.
Cache	A temporary copy of data kept in fast storage so it can be read quickly; it can always be rebuilt from the original.
CMC	Chinese Multi-Cinema, the client organization.
Customer account	An optional account storing a customer's profile, purchase history, loyalty points, membership, and payment tokens.
Digital ticket	A ticket delivered electronically that carries a unique machine-readable validation code.
Idempotency key	A unique value sent with a request so that repeating the request has no extra effect (for example, a payment is not charged twice).
MoSCoW	Priority scheme used in this SRS: Must have, Should have, Could have (see Section 3).
Payment token	A reference issued by the payment processor that stands in for card details; CMC never stores the card number itself.
PCI DSS	Payment Card Industry Data Security Standard.
Showtime	A scheduled screening of a movie in a specific auditorium at a specific date and time.
Replica	A copy of a database that is kept up to date automatically from the main (primary) database.
Schema (database)	A named group of related tables inside one database.
SQL / NoSQL	SQL databases store data in related tables queried with the SQL language; NoSQL databases use other models, such as documents or key-value pairs.
SRS	Software Requirements Specification (this document).
Ticket hold	A temporary five-minute reservation of selected tickets or seats created when checkout begins.
TLS	Transport Layer Security, the encryption used by HTTPS connections.
Transaction	A completed or attempted purchase, refund, or override recorded by the system. (A *database transaction* is a group of database changes saved all together or not at all.)
UR / FR / NFR / UC	Prefixes for User Requirements, Functional Requirements, Non-Functional Requirements, and Use Cases.

1.4 References

- IEEE Computer Society. *IEEE Recommended Practice for Software Requirements Specifications*, IEEE Std 830-1998. IEEE, 1998. Available from IEEE Xplore (ieeexplore.ieee.org).
- Object Management Group. *OMG Unified Modeling Language (UML)*, Version 2.5.1. OMG, December 2017. Available from omg.org/spec/UML.

- World Wide Web Consortium. *Web Content Accessibility Guidelines (WCAG) 2.1*, W3C Recommendation. Available from w3.org/TR/WCAG21.
- PCI Security Standards Council. *Payment Card Industry Data Security Standard*, Version 4.0. March 2022. Available from pcisecuritystandards.org.
- Software Engineering Tutorial (course reading), CS 250 - Introduction to Software Systems, Fall 2026.
- Atlassian. *How to Write a Test Plan*. Available from atlassian.com/software/confluence/resources/guides/how-to/test-plan.
- GeeksforGeeks. *Test Plan - Software Testing*. Available from geeksforgeeks.org/software-testing/test-plan-software-testing/.
- The PostgreSQL Global Development Group. *PostgreSQL Documentation*. Available from postgresql.org/docs.
- Redis Ltd. *Redis Documentation*. Available from redis.io/docs.

1.5 Overview

The rest of this SRS moves from general to specific. **Section 2 - General Description** is written for non-technical readers: it explains the product's context, the user requirements (what each type of user needs to accomplish), the characteristics of those users, and the constraints, assumptions, and dependencies that shape the requirements. **Section 3 - Specific Requirements** contains the detailed, numbered requirements used by developers and testers: external interfaces (3.1), functional requirements grouped by feature (3.2), the use-case diagram and use-case descriptions (3.3), the main classes and objects (3.4), and measurable non-functional requirements (3.5). **Section 4 - Software Design Specification** describes how the system would be built: a system description (4.1), the software architecture (4.2), the UML class diagram (4.3), the classes with their attributes and operations (4.4), and the development plan (4.5), and the design updates made in this version (4.6). **Section 5 - Test Plan** describes how the system will be verified and validated: the test strategy (5.1-5.2), the test scope drawn on the architecture (5.3), the unit, integration, functional, and system test sets (5.4-5.7), the sample test cases (5.8), requirements traceability (5.9), and the test schedule (5.10). **Section 6 - Design 2.0: Architecture and Data Management** updates the design in Section 4: the updated architecture diagram with every data store (6.1), the changes made since the original design (6.2), the data management strategy, including the logical data model, data integrity, security, and backups (6.3), and the design decisions, alternatives, and tradeoffs behind it (6.4).

2. General Description

This section describes the general factors that affect the CMC Ticketing System and its requirements. It does not state specific requirements; it provides the background that makes the requirements in Section 3 easier to understand.

2.1 Product Perspective

The CMC Ticketing System replaces CMC's existing ticketing software, which provides basic sales but has become slow and difficult to maintain. Unlike the current system, the new product gives all 20 theaters and all three sales channels, the public website, lobby kiosks, and box-office workstations, one shared source of movie, showtime, price, seat, and transaction information. A ticket sold at one theater's box office is immediately unavailable on the website, and vice versa.

The system is self-contained but depends on three external services: a **payment service** that authorizes and refunds card and PayPal payments, an **email service** that delivers confirmations and digital tickets, and a licensed **movie review service** that supplies audience and critic scores. It also works with the ticket printers and code scanners already installed in CMC theaters.

2.2 Product Functions (User Requirements)

The following user requirements summarize, in non-technical terms, what each group of users must be able to do. The last column traces each one to the functional requirements (Section 3.2) and use cases (Section 3.3) that refine it.

ID	User requirement	Users	Refined by
UR-01	Browse movies, showtimes, prices, seat availability, and review scores at any CMC theater.	Customer, Employee	FR-01-06; UC-01
UR-02	Buy up to 20 tickets for a showtime in one purchase, online or at a kiosk.	Customer	FR-07, 08, 10, 11, 13, 15-21; UC-02
UR-03	Choose specific seats for deluxe showings; buy general-admission tickets for regular showings.	Customer	FR-09, 12; UC-03
UR-04	Buy tickets as a guest or with an optional account.	Customer	FR-22, 23; UC-02, UC-05
UR-05	Receive tickets by email or print them.	Customer	FR-21; UC-02
UR-06	View purchase history, loyalty points, and CMC membership status in an account.	Account holder	FR-24-28; UC-05
UR-07	Give quick feedback on the purchase experience.	Customer	FR-44; UC-04
UR-08	Sell and print tickets for walk-up customers at the box office.	Theater employee	FR-29; UC-06
UR-09	Scan tickets at the entrance and admit each ticket only once.	Theater employee	FR-30-32; UC-07
UR-10	Refund eligible tickets in person before the showtime starts.	Theater employee	FR-30, 33; UC-08
UR-11	Manage movies, showtimes, auditoriums, and ticket prices.	Administrator	FR-14, 34-37, 39, 40; UC-09
UR-12	Correct customer purchasing mistakes with a recorded reason.	Administrator	FR-38, 39; UC-10
UR-13	See tickets sold and revenue by theater, movie, showtime, and date.	CMC management	FR-41-43; UC-11
UR-14	Get fair access to tickets during high-demand releases, without bots buying them first.	Customer	FR-45, 46; UC-02

2.3 User Characteristics

User class	Characteristics	Needs that affect the requirements
Guest customer	Members of the public of any age and technical skill; occasional use; may prefer English or Spanish.	A short, simple purchase flow; clear prices before paying; no forced registration.
Account holder	Returning customers who buy frequently.	Saved history, loyalty points, membership benefits, and saved (tokenized) payment methods.
Kiosk customer	Uses a shared public touchscreen, often in a hurry.	Large touch targets, very few steps, and automatic clearing of personal data.
Theater employee	Box-office and usher staff; trained on the system; frequent daily use.	Fast selling, scanning, lookup, and refund tools that work under time pressure.
Administrator	Trained CMC operations staff with elevated permissions.	Secure access to scheduling, pricing, and override functions with an audit trail.
CMC management	Business users who are not technical.	Summarized, filterable sales and revenue reports across all theaters.

2.4 General Constraints

- The customer-facing system must run in a standard web browser; no native mobile application is required for the first release.
- The initial deployment serves 20 CMC theaters in the San Diego area, operating in Pacific Time and in U.S. dollars.
- Business rules set by CMC: at most 20 tickets per transaction; sales open 14 days before a showtime and close 10 minutes after it starts; a checkout hold lasts five minutes; regular auditoriums have 150 general-admission seats and deluxe auditoriums have 75 assigned seats.
- Prices and availability must be identical on every sales channel.

2.5 Assumptions and Dependencies

- Every CMC theater has a reliable internet connection. If this is not true, offline box-office sales would become a new requirement.
- Authorized employees enter accurate movie, auditorium, showtime, and price information.
- An approved third-party payment service is available for card and PayPal payments (and for Bitcoin, if CMC enables it).
- An external email service is available to deliver confirmations and digital tickets.
- CMC obtains a license or API agreement with a movie review provider; without it, review scores are omitted (FR-06 is a Should-have).
- Customers use a currently supported web browser (see NFR-PT-01).
- The ticket printers and code scanners at each theater are working and support standard barcode/QR formats.

3. Specific Requirements

This section contains the detailed requirements used to guide design, implementation, and testing. Each requirement has a unique ID, uses "shall" for anything mandatory, can be checked by testing or inspection, and traces back to a user requirement in Section 2.2.

Functional requirements carry a priority based on the MoSCoW scheme from the course reading (*Software Engineering Tutorial*): **Must** - the system is not operational without it; **Should** - important and expected in the first release; **Could** - desirable and included if time and budget allow.

3.1 External Interface Requirements

3.1.1 User Interfaces

ID	Requirement
UI-01	Before payment, the system shall display the movie, theater, date, showtime, auditorium type, ticket types and quantities, assigned seats (if any), fees, and total cost on one screen.
UI-02	The deluxe seat map shall visually distinguish available, held, selected, and sold seats, using both color and a text or symbol label.
UI-03	During checkout the interface shall continuously display the time remaining in the five-minute ticket hold.
UI-04	All customer-facing screens shall be available in English and Spanish, selectable from every page.
UI-05	The kiosk interface shall use touch targets of at least 1.5 cm $\times$ 1.5 cm and shall complete a standard purchase in no more than six screens.

ID	Requirement
UI-06	Employee and administrator functions shall be reachable only after sign-in and shall never be shown on customer-facing screens.

3.1.2 Hardware Interfaces

ID	Requirement
HI-01	The system shall run on customer computers and phones, theater kiosks, and employee workstations that have a supported web browser (NFR-PT-01); no special hardware shall be required.
HI-02	The system shall produce printable tickets and receipts compatible with CMC's existing ticket and receipt printers.
HI-03	Each ticket shall carry a machine-readable code (QR code or barcode) readable by CMC's existing ticket scanners.
HI-04	The kiosk interface shall support touchscreen input without a keyboard or mouse.

3.1.3 Software Interfaces

ID	Requirement
SI-01	The system shall send payment authorization and refund requests to an approved payment service and store only the returned status, transaction reference, and payment token.
SI-02	The system shall obtain movie review scores and critic information only through a licensed API from an approved review provider, not by copying content from websites.
SI-03	The website, kiosks, and employee workstations shall read and update the same centralized ticket, seat, and price data.
SI-04	The system shall send confirmations and digital tickets through an approved email delivery service and record each message's delivery status.

3.1.4 Communications Interfaces

ID	Requirement
CI-01	All communication between clients and the server shall use HTTPS with TLS 1.2 or higher.
CI-02	Payment information shall be sent only to the approved payment service, over an encrypted connection.
CI-03	Purchase confirmations and digital tickets shall be sent to the email address entered at checkout within two minutes of payment approval.

3.2 Functional Requirements

Functional requirements are grouped by feature. Each feature follows the template structure of introduction, inputs, processing (the numbered requirements), outputs, and error handling.

3.2.1 Browse Movies and Showtimes

3.2.1.1 Introduction. Lets customers and employees find what is playing, when and where, what it costs, and how many seats remain.

3.2.1.2 Inputs. Theater, movie, date, and preferred language.

3.2.1.3 Processing.

ID	Requirement	Priority
FR-01	The system shall display the movies scheduled at a selected CMC theater for a selected date.	Must
FR-02	For each showtime the system shall display the date, start time, auditorium type, price for each ticket type, and number of seats remaining.	Must
FR-03	The system shall let users filter showtimes by theater, movie, and date.	Must
FR-04	The system shall open ticket sales for a showtime exactly 14 calendar days before its start time and shall not sell tickets earlier.	Must
FR-05	The system shall close ticket sales for a showtime 10 minutes after its scheduled start time.	Must
FR-06	The system shall display audience and critic scores for each movie obtained from the licensed review service.	Should

3.2.1.4 Outputs. A list of matching movies and showtimes with auditorium type, prices, remaining availability, and review scores.

3.2.1.5 Error Handling. If no showtimes match the criteria, the system shall display a "no showtimes found" message and keep the user's filters so they can be changed. If the review service does not respond within two seconds, the system shall show the showtimes without review scores; browsing and purchasing shall continue normally.

3.2.2 Select and Hold Tickets

3.2.2.1 Introduction. Lets a customer choose ticket types and quantities (and seats, for deluxe showings) and reserves them while the customer pays.

3.2.2.2 Inputs. Showtime, ticket types (ADULT, CHILD, STUDENT, SENIOR, MILITARY), quantities, and deluxe seat selections.

3.2.2.3 Processing.

ID	Requirement	Priority
FR-07	The system shall let the customer select a quantity for each ticket type offered for the showtime.	Must
FR-08	The system shall reject any selection that would place more than 20 tickets in one transaction.	Must
FR-09	For deluxe showtimes the system shall display a seat map and require the customer to choose one available seat per ticket.	Must
FR-10	For regular showtimes the system shall sell general-admission tickets and shall never sell more tickets than the auditorium's capacity.	Must
FR-11	When checkout begins the system shall hold the selected tickets and seats for five minutes.	Must
FR-12	The system shall prevent a held or sold seat from being selected by any other customer on any channel.	Must
FR-13	The system shall release held tickets and seats automatically when the hold expires without an approved payment.	Must
FR-14	The system shall price ticket types according to the rules configured by administrators (FR-37).	Must

3.2.2.4 Outputs. An order summary listing the selected tickets, seats (if any), fees, total price, and hold expiration time.

3.2.2.5 Error Handling. If a selected seat or ticket becomes unavailable before the hold is created, the system shall identify it and ask the customer to choose again. When the hold expires, the system shall tell the customer, release the inventory, and allow the customer to restart the selection.

3.2.3 Complete Purchase and Deliver Tickets

3.2.3.1 Introduction. Collects payment for held tickets and issues valid tickets only after payment is approved.

3.2.3.2 Inputs. Held order, customer email address, payment method, and payment details (entered on the payment service's secure form).

3.2.3.3 Processing.

ID	Requirement	Priority
FR-15	The system shall display an itemized total (UI-01) and require the customer to confirm it before payment is submitted.	Must
FR-16	The system shall accept credit cards, debit cards, and PayPal.	Must
FR-17	The system shall accept Bitcoin through an approved cryptocurrency payment processor.	Could
FR-18	The system shall not issue valid tickets until the payment service approves the payment.	Must
FR-19	The system shall create exactly one transaction record, with a unique confirmation number, for each approved payment.	Must
FR-20	The system shall generate a unique, machine-readable validation code for every ticket issued.	Must
FR-21	The system shall email the tickets and receipt to the customer and offer a printable version on the confirmation screen.	Must

3.2.3.4 Outputs. A confirmation screen with confirmation number and receipt; digital tickets by email; printed tickets on request.

3.2.3.5 Error Handling. If payment is declined, the system shall display the reason, issue no tickets, and let the customer try another payment method while the hold remains active. If the payment service's response is unknown (e.g., a timeout), the system shall query the transaction status before allowing another attempt, so the customer is never charged twice. If email delivery fails, tickets shall remain available from the confirmation screen, the customer account, or the box office.

3.2.4 Customer Accounts and Membership

3.2.4.1 Introduction. Provides optional accounts for returning customers, including purchase history, loyalty points, and CMC membership.

3.2.4.2 Inputs. Name, email address, password, membership selection, and saved payment choices.

3.2.4.3 Processing.

ID	Requirement	Priority
FR-22	The system shall let customers purchase tickets as guests by providing only an email address and payment.	Must
FR-23	The system shall let customers create an optional account using an email address and password.	Should
FR-24	The system shall let account holders view their past and upcoming purchases and re-send or print their tickets.	Should
FR-25	The system shall award loyalty points on each purchase at the rate configured by CMC and display the current balance.	Should
FR-26	The system shall let account holders save payment methods, storing only payment tokens (NFR-S-03).	Should
FR-27	The system shall allow only one active sign-in session per account; signing in on a new device shall end the previous session.	Must
FR-28	The system shall let account holders purchase a CMC membership and apply its benefits at checkout.	Could

3.2.4.4 Outputs. An updated profile, purchase history, loyalty balance, or membership status.

3.2.4.5 Error Handling. The system shall reject registration with an email address already in use and shall reject invalid sign-in credentials with a message that does not reveal which field was wrong. An expired membership shall not be applied at checkout; the customer shall be told it has expired.

3.2.5 Box Office Sales, Ticket Validation, and Refunds

3.2.5.1 Introduction. Supports theater employees selling tickets in person, admitting customers, and handling refunds.

3.2.5.2 Inputs. Employee credentials, showtime and ticket selections, ticket codes (scanned or typed), confirmation numbers or customer email addresses, and refund reasons.

3.2.5.3 Processing.

ID	Requirement	Priority
FR-29	The system shall let employees sell and print tickets for walk-up customers using the same inventory, prices, and rules as online sales.	Must
FR-30	The system shall let employees find a transaction by confirmation number, ticket code, or customer email address.	Must
FR-31	The system shall validate a ticket from its scanned or typed code and display one of: valid, already used, refunded, wrong showtime, or not found.	Must
FR-32	The system shall mark a ticket as redeemed on its first successful validation and reject every later attempt.	Must
FR-33	The system shall let authorized employees refund eligible tickets in person before the showtime starts, invalidate the refunded tickets, and return their seats to inventory.	Must

3.2.5.4 Outputs. Printed tickets and receipts, validation results, and refund confirmations.

3.2.5.5 Error Handling. The system shall reject refunds requested after the showtime has started and refunds of tickets that have been redeemed or already refunded. If the payment service rejects a refund, the ticket shall remain valid and the employee shall be shown the reason.

3.2.6 Administration

3.2.6.1 Introduction. Lets administrators maintain the movie, showtime, auditorium, and price data and record each change.

3.2.6.2 Inputs. Administrator credentials; movie, showtime, auditorium, and price data; override reasons.

3.2.6.3 Processing.

ID	Requirement	Priority
FR-34	The system shall let administrators create, modify, and cancel movies and showtimes.	Must
FR-35	The system shall assign each showtime to an auditorium and reject showtimes that overlap in the same auditorium.	Must
FR-36	The system shall let administrators configure each auditorium's type (regular or deluxe), capacity, and seat layout.	Must
FR-37	The system shall let administrators configure ticket prices (ticket type and amount)	Must
FR-38	The system shall let administrators correct a customer's purchase (e.g., wrong showtime or ticket type) through an override that requires a written reason.	Should
FR-39	The system shall record the employee ID, action, reason, and timestamp for every refund, override, and price or schedule change.	Must

ID	Requirement	Priority
FR-40	When a showtime with sold tickets is cancelled, the system shall mark all affected tickets as refund-eligible and list them for follow-up.	Should

3.2.6.4 Outputs. Updated schedules, prices, and auditorium settings, available on all channels within two seconds; audit records.

3.2.6.5 Error Handling. The system shall reject overlapping showtimes, negative or missing prices, and overrides without a reason, and shall explain why. Unauthorized users attempting administrative functions shall be denied and the attempt shall be logged (NFR-S-04).

3.2.7 Reporting, Feedback, and Demand Protection

3.2.7.1 Introduction. Gives management visibility into sales, collects customer feedback, and keeps the system fair and responsive during high-demand releases.

3.2.7.2 Inputs. Report filters (date range, theater, movie, showtime), completed transactions, feedback ratings, and traffic measurements.

3.2.7.3 Processing.

ID	Requirement	Priority
FR-41	The system shall record every completed purchase and refund in a system-wide daily transaction log.	Must
FR-42	The system shall let management users view tickets sold and revenue by theater, movie, showtime, and date range.	Must
FR-43	The system shall let management users export any report as a CSV file.	Could
FR-44	After a completed purchase the system shall offer an optional 1-5 satisfaction rating and store it with the transaction.	Could
FR-45	When checkout demand exceeds capacity, the system shall place new customers in a first-come, first-served waiting queue and show their position.	Should
FR-46	The system shall detect and block automated (bot) purchasing, as specified in NFR-S-08.	Should

3.2.7.4 Outputs. Sales and revenue reports, CSV exports, stored feedback, and queue-position messages.

3.2.7.5 Error Handling. If no records match a report's filters, the system shall display an empty report rather than an error. If report generation fails, transaction data shall remain unchanged and the user shall be notified. A customer in the queue shall keep their place if the page is refreshed.

3.3 Use Cases

Figure 1 shows every actor and use case of the CMC Ticketing System. Actors on the left are people who start use cases; actors on the right are external systems that the CMC system calls on. A solid line means the actor takes part in the use case. A dashed «extend» arrow means the extending use case adds optional behavior to the base use case under a stated condition. A text description of each use case follows the figure.

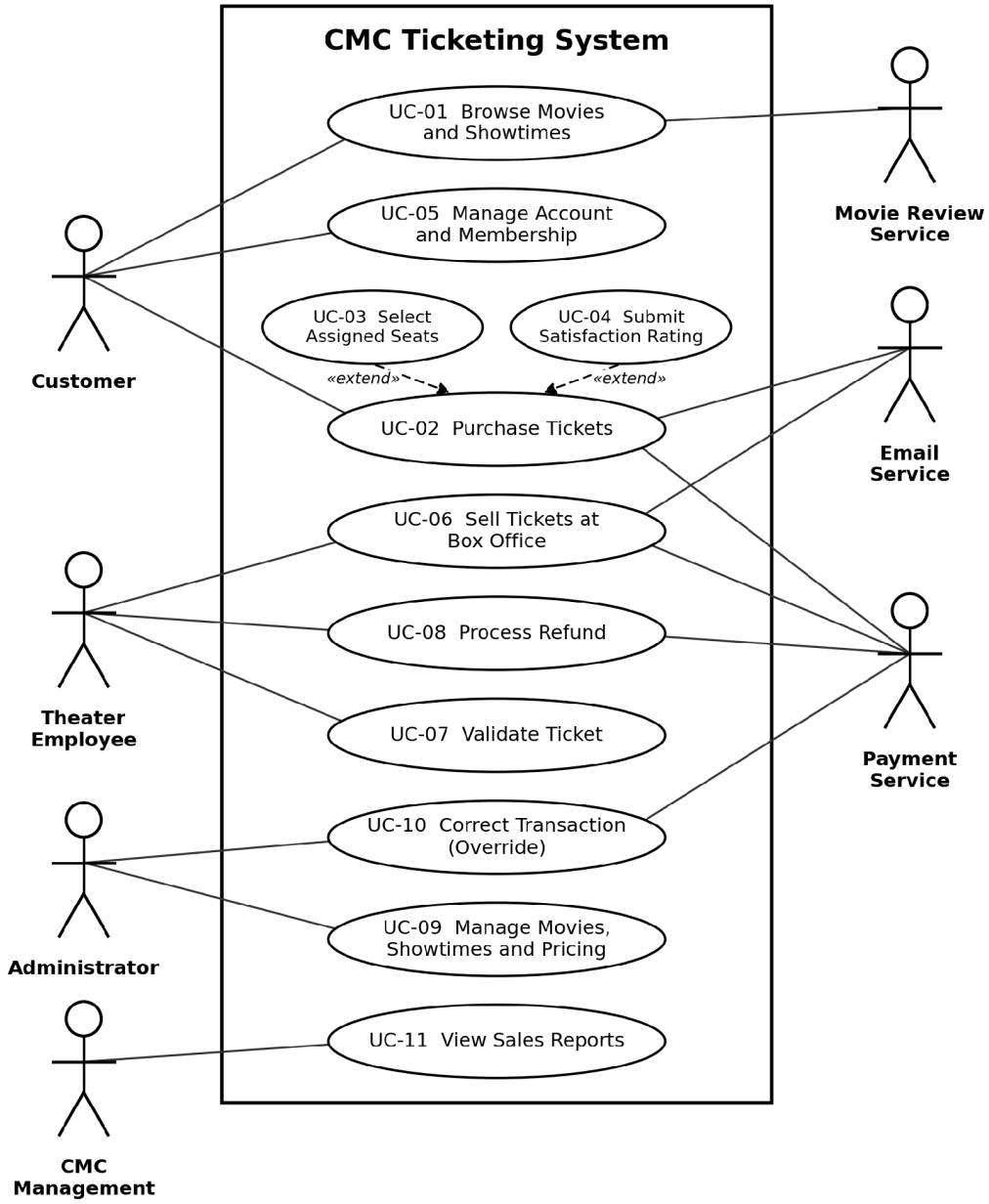


Figure 1. Use-case diagram for the CMC Ticketing System

3.3.1 Use Case #1 (UC-01): Browse Movies and Showtimes

Description	A customer looks up what is playing at a CMC theater and picks a showtime.
Actors	Primary: Customer. Secondary: Movie Review Service.
Related requirements	FR-01 - FR-06; UR-01
Precondition / trigger	At least one showtime has been scheduled by an administrator. *Trigger:* The customer opens the CMC website or kiosk.

Description	A customer looks up what is playing at a CMC theater and picks a showtime.
Main success scenario	1. The customer selects a theater and date. 2. The system displays the movies and showtimes for that theater and date. 3. The customer selects a movie. 4. The system displays each showtime's time, auditorium type, prices, seats remaining, and review scores. 5. The customer selects a showtime to buy tickets (continues in UC-02).
Alternate / exception flows	**2a.** No showtimes match: the system shows a no-results message and the customer changes the filters. **4a.** The review service is unavailable: the system shows the showtimes without scores. **5a.** The showtime is outside its sales window: the system shows when sales open or that sales have closed.
Postconditions	No data is changed; the customer has either chosen a showtime or left.

3.3.2 Use Case #2 (UC-02): Purchase Tickets

Description	A customer buys tickets online or at a kiosk and receives them by email or print.
Actors	Primary: Customer. Secondary: Payment Service, Email Service.
Related requirements	FR-07 - FR-22, FR-45, FR-46; UR-02, UR-04, UR-05, UR-14
Precondition / trigger	A showtime inside its sales window has been selected (UC-01). *Trigger:* The customer chooses "Buy tickets".
Main success scenario	1. The system displays the available ticket types and prices. 2. The customer selects ticket types and quantities (20 or fewer). 3. **Extension point - seat selection:** if the showing is deluxe, UC-03 is performed. 4. The system holds the selection for five minutes and starts the countdown. 5. The customer continues as a guest (email address) or signs in. 6. The system displays the itemized total; the customer confirms and enters payment. 7. The payment service approves the payment. 8. The system records one transaction, issues tickets with unique codes, and removes them from inventory. 9. The system displays the confirmation and emails the tickets and receipt. 10. **Extension point - feedback:** UC-04 may be performed.
Alternate / exception flows	**2a.** More than 20 tickets requested: the system blocks the selection and explains the limit. **4a.** High demand: the customer is placed in the waiting queue and continues when a checkout slot is free. **7a.** Payment declined: no tickets are issued; the customer may try another method while the hold is active. **7b.** Hold expires before approval: the system releases the inventory and asks the customer to start again. **9a.** Email fails: tickets remain printable from the confirmation screen and retrievable at the box office.
Postconditions	Payment is captured once, tickets are valid, and the purchased seats are unavailable on every channel.

3.3.3 Use Case #3 (UC-03): Select Assigned Seats (extends UC-02)

Description	For a deluxe showing, the customer chooses specific seats on a seat map.
Actors	Primary: Customer (through UC-02). Secondary: None.
Related requirements	FR-09, FR-12; UR-03
Precondition / trigger	UC-02 is in progress for a deluxe showtime. *Trigger:* UC-02 reaches the seat-selection extension point.
Main success scenario	1. The system displays the auditorium seat map with available, held, and sold seats. 2. The customer selects one seat per ticket. 3. The system confirms that each seat is still available. 4. Control returns to UC-02 step 4.
Alternate / exception flows	**3a.** A seat was taken by another customer: the system marks it unavailable and asks the customer to choose another.
Postconditions	The chosen seats are ready to be held for this customer.

3.3.4 Use Case #4 (UC-04): Submit Satisfaction Rating (extends UC-02)

Description	After a purchase, the customer optionally rates the experience.
Actors	Primary: Customer (through UC-02). Secondary: None.
Related requirements	FR-44; UR-07
Precondition / trigger	UC-02 has completed successfully. *Trigger:* The confirmation screen displays the rating control.
Main success scenario	1. The customer selects a rating from 1 to 5. 2. The system stores the rating with the transaction and thanks the customer.
Alternate / exception flows	**1a.** The customer ignores or closes the rating control: nothing is stored.
Postconditions	The rating, if given, is linked to the transaction.

3.3.5 Use Case #5 (UC-05): Manage Account and Membership

Description	A customer creates or maintains an optional account.
Actors	Primary: Customer (account holder). Secondary: None.
Related requirements	FR-23 - FR-28; UR-04, UR-06
Precondition / trigger	None for registration; an existing account for all other actions. *Trigger:* The customer selects "Create account" or "My account".
Main success scenario	1. The customer registers or signs in. 2. The system ends any other active session for the account. 3. The system displays purchase history, upcoming tickets, loyalty points, and membership status. 4. The customer updates the profile, removes a saved payment method, or buys a membership. 5. The system validates and saves the change.
Alternate / exception flows	**1a.** Email already registered or invalid credentials: the request is rejected with a general message. **4a.** Membership payment declined: membership status is unchanged.
Postconditions	Account data is saved; no card numbers are stored.

3.3.6 Use Case #6 (UC-06): Sell Tickets at Box Office

Description	An employee sells tickets to a walk-up customer.
Actors	Primary: Theater employee. Secondary: Payment Service, Email Service.
Related requirements	FR-29, FR-07 - FR-21; UR-08
Precondition / trigger	The employee is signed in at a box-office workstation. *Trigger:* A customer asks to buy tickets at the counter.
Main success scenario	1. The employee selects the showtime, ticket types, and quantities (and seats for deluxe showings). 2. The system holds the selection. 3. The employee takes payment through the payment service. 4. The system records the transaction and issues tickets. 5. The employee prints the tickets and, if requested, emails them to the customer.
Alternate / exception flows	**2a.** Tickets were just sold on another channel: the system shows current availability and the employee re-selects. **3a.** Payment declined: no tickets are issued.
Postconditions	The sale is recorded and inventory is updated on every channel.

3.3.7 Use Case #7 (UC-07): Validate Ticket

Description	An employee admits a customer by scanning the ticket.
Actors	Primary: Theater employee. Secondary: None (ticket scanner is a device used by the employee).
Related requirements	FR-30 - FR-32; UR-09
Precondition / trigger	The employee is signed in; the ticket was issued for a showtime at this theater. *Trigger:* The customer presents a printed or digital ticket.
Main success scenario	1. The employee scans the ticket code (or types it). 2. The system checks the ticket's showtime, status, and validation history. 3. The system marks the ticket as redeemed. 4. The system displays "valid" and the auditorium and seat.
Alternate / exception flows	**2a.** Unknown, refunded, wrong-showtime, or already-used ticket: the system displays the reason and does not mark it redeemed.
Postconditions	A valid ticket has been redeemed exactly once.

3.3.8 Use Case #8 (UC-08): Process Refund

Description	An employee refunds eligible tickets in person.
Actors	Primary: Theater employee. Secondary: Payment Service.
Related requirements	FR-30, FR-33, FR-39; UR-10
Precondition / trigger	The employee is signed in with refund permission; the showtime has not started. *Trigger:* A customer asks for a refund at the box office.
Main success scenario	1. The employee finds the transaction by confirmation number, ticket code, or email. 2. The system shows the tickets and which are eligible for refund. 3. The employee selects tickets and enters a reason. 4. The system sends the refund to the payment service. 5. The system invalidates the refunded tickets, returns the seats to inventory, and records the audit entry. 6. The system prints or emails a refund confirmation.
Alternate / exception flows	**2a.** Showtime already started, or ticket already used or refunded: the refund is refused. **4a.** Payment service rejects the refund: tickets stay valid and the employee sees the reason.
Postconditions	The refund and its audit record are stored, and refunded seats are available for sale.

3.3.9 Use Case #9 (UC-09): Manage Movies, Showtimes and Pricing

Description	An administrator maintains the schedule, auditoriums, and prices.
Actors	Primary: Administrator. Secondary: None.
Related requirements	FR-34 - FR-37, FR-39, FR-40; UR-11
Precondition / trigger	The administrator is signed in with administrative permission. *Trigger:* The administrator opens the admin page.
Main success scenario	1. The administrator selects movies, showtimes, auditoriums, or prices. 2. The system displays the current values. 3. The administrator enters changes. 4. The system checks for overlapping showtimes, capacity conflicts, and invalid prices. 5. The administrator confirms. 6. The system saves the change, records an audit entry, and publishes it to all channels.
Alternate / exception flows	**4a.** Conflict or invalid value: the change is rejected with an explanation. **6a.** A cancelled showtime has sold tickets: the affected tickets are marked refund-eligible and listed for follow-up.

Description	An administrator maintains the schedule, auditoriums, and prices.
Postconditions	Valid changes are live on every channel and recorded in the audit log.

3.3.10 Use Case #10 (UC-10): Correct Transaction (Override)

Description	An administrator fixes a customer's purchasing mistake.
Actors	Primary: Administrator. Secondary: Payment Service.
Related requirements	FR-38, FR-39; UR-12
Precondition / trigger	The administrator is signed in; the transaction exists and its showtime has not started. *Trigger:* A customer or employee reports a purchasing error.
Main success scenario	1. The administrator finds the transaction. 2. The administrator selects the correction (e.g., change showtime or ticket type) and enters a reason. 3. The system calculates any price difference and, if needed, charges or refunds it through the payment service. 4. The system reissues the corrected tickets, invalidates the old ones, and records the audit entry.
Alternate / exception flows	**2a.** No reason entered: the override is refused. **3a.** The price-difference payment fails: no change is made.
Postconditions	The customer holds corrected, valid tickets; the original tickets are invalid; the override is audited.

3.3.11 Use Case #11 (UC-11): View Sales Reports

Description	A CMC manager reviews ticket sales and revenue.
Actors	Primary: CMC management. Secondary: None.
Related requirements	FR-41 - FR-43; UR-13
Precondition / trigger	The user is signed in with reporting permission. *Trigger:* The user opens the reports page.
Main success scenario	1. The user selects a date range and, optionally, theaters, movies, or showtimes. 2. The system retrieves the matching transactions. 3. The system calculates tickets sold and revenue, minus refunds. 4. The system displays the report, with an option to export it.
Alternate / exception flows	**2a.** No matching records: an empty report is shown. **3a.** Report generation fails: the user is notified; data is unchanged.
Postconditions	The report is displayed; no transaction data is modified.

3.4 Classes / Objects

The following conceptual classes represent the main information the system manages. They describe required data and responsibilities without prescribing a programming language or final design. These class descriptions use several enumerated types (Enums), which are data types that can only take one value from a fixed set of allowed options. Named Enums such as `SeatStatus` and `TicketStatus` are used to keep attribute types clear and consistent.

Enumerated Types

- AuditoriumType = {REGULAR, DELUXE}
- ShowtimeStatus = {SCHEDULED, ON_SALE, CLOSED, CANCELLED}
- SeatStatus = {AVAILABLE, HELD, SOLD}

- TicketType = {ADULT, CHILD, STUDENT, SENIOR, MILITARY}
- TicketStatus = {ISSUED, REDEEMED, REFUNDED}
- TransactionType = {PURCHASE, REFUND, OVERRIDE}
- TransactionStatus = {APPROVED, DECLINED, REFUNDED, VOIDED}
- SalesChannel = {WEB, KIOSK, BOX_OFFICE}
- MembershipType = {NONE, MEMBER}
- Role = {BOX_OFFICE, ADMINISTRATOR, MANAGER}

3.4.1 Theater

A CMC theater location.

3.4.1.1 Attributes:

- theaterID : Integer
- name : String
- address : String
- phoneNumber : String
- auditoriums : String[]

3.4.1.2 Functions:

- getShowtimes(date) : String[] - returns the showtimes scheduled at this theater
- getAuditoriums() : String[] - returns the theater's auditoriums.

Related requirements and use cases: FR-01, FR-03, FR-42; UC-01, UC-11

3.4.2 Auditorium

A screening room within a theater.

3.4.2.1 Attributes:

- auditoriumID : Integer
- theaterID : Integer
- type (Regular or Deluxe) : AuditoriumType
- capacity (150 or 75) : Integer
- seatLayout (row and number of each deluxe seat) : String[][]

3.4.2.2 Functions:

- isDeluxe() : Boolean - tells whether seats are assigned
- getSeatLayout() : String[][] - returns the seats in the room.

Related requirements and use cases: FR-09, FR-10, FR-36; UC-03, UC-09

3.4.3 Movie

A film that CMC is showing.

3.4.3.1 Attributes:

- movieID : Integer
- title : String
- runtime : Integer
- mpaaRating : String
- synopsis : String
- audienceCriticScores (from the review service) : Integer[]

3.4.3.2 Functions:

- getReviewScores() : Integer[] - retrieves current scores from the review service.

Related requirements and use cases: FR-01, FR-06, FR-34; UC-01, UC-09

3.4.4 Showtime

One screening of a movie in an auditorium, including the status of every seat for that screening.

3.4.4.1 Attributes:

- showtimeID : Integer
- movieID : Integer
- auditoriumID : Integer
- startDateAndTime : DateTime
- status (Scheduled, On Sale, Closed, Cancelled) : ShowtimeStatus
- salesOpen (start − 14 days) : DateTime
- salesClose (start + 10 minutes) : DateTime
- seatsRemaining : Integer
- deluxeSeatStatus (seat label -> Available, Held, Sold) : `Map<String,SeatStatus>`

3.4.4.2 Functions:

- isOnSale(now) : Boolean - applies the sales window
- holdSeats(), releaseSeats(), markSold() : void - change seat status so only one customer can hold or buy a seat.
- cancel() : void - cancels the showtime and flags affected tickets.

Related requirements and use cases: FR-02, FR-04, FR-05, FR-09, FR-12, FR-13, FR-34, FR-35, FR-40; UC-01, UC-03, UC-09

3.4.5 TicketHold

A temporary reservation made when checkout begins.

3.4.5.1 Attributes:

- holdID : Integer
- showtimeID : Integer
- selectedTicketsAndSeats : String[]
- createdAt : DateTime
- expiresAt (createdAt + 5 minutes) : DateTime

3.4.5.2 Functions:

- isExpired() : Boolean - true after five minutes.
- release() : void - returns tickets and seats to inventory.

Related requirements and use cases: FR-11, FR-13; UC-02, UC-06

3.4.6 Ticket

Admission for one person to one showtime.

3.4.6.1 Attributes:

- ticketID : Integer
- validationCode : String
- showtimeID : Integer
- seat : String
- ticketType (ADULT, CHILD, STUDENT, SENIOR, MILITARY) : TicketType
- pricePaid : Decimal
- status (Issued, Redeemed, Refunded) : TicketStatus

3.4.6.2 Functions:

- validate() : Boolean - redeems the ticket once and rejects later attempts;
- refund() : void - marks the ticket refunded and frees its seat.

Related requirements and use cases: FR-20, FR-31 - FR-33; UC-07, UC-08

3.4.7 Transaction

A purchase, refund, or override.

3.4.7.1 Attributes:

- transactionID : Integer
- confirmationNumber : Integer
- type (Purchase, Refund, Override) : TransactionType
- tickets : Ticket[]
- subtotal : Decimal
- fees : Decimal
- total : Decimal
- paymentToken : String

- status : TransactionStatus
- salesChannel (Web, Kiosk, Box Office) : SalesChannel
- timestamp : DateTime
- satisfactionRating (1-5) : Integer

3.4.7.2 Functions:

- authorizePayment() : void - requests approval from the payment service
- issueTickets() : void - creates tickets after approval
- getReceipt() : void - produces the receipt.

Related requirements and use cases: FR-15 - FR-19, FR-29, FR-41, FR-44; UC-02, UC-04, UC-06, UC-08, UC-10

3.4.8 CustomerAccount

An optional customer account.

3.4.8.1 Attributes:

- customerID : Integer
- name : String
- email : String
- passwordHash : String
- savedPaymentTokens : String
- loyaltyPointBalance : Integer
- membershipType (None or Member) : MembershipType
- membershipNumber : Integer
- activeSessionID : Integer

3.4.8.2 Functions:

- signIn() : void - starts a session and ends any previous one
- getPurchaseHistory() : String[] - lists past and upcoming purchases
- addLoyaltyPoints(transaction) : Integer - awards points.

Related requirements and use cases: FR-22 - FR-28; UC-05

3.4.9 EmployeeAccount

A CMC staff account.

3.4.9.1 Attributes:

- employeeID : Integer
- name : String
- home theater : Integer
- role (Box Office, Administrator, Manager) : Role
- passwordHash: String

3.4.9.2 Functions:

- hasPermission(action) : Boolean - checks role-based access before any employee function.

Related requirements and use cases: FR-29 - FR-42; UC-06 - UC-11

3.4.10 AuditRecord

A permanent record of a sensitive employee action.

3.4.10.1 Attributes:

- recordID : Integer
- employeeID : Integer
- action : String
- reason : String
- relatedData : String - the data that the audit pertains to
- timestamp : DateTime

3.4.10.2 Functions:

- create() : void - records are write-once and cannot be edited or deleted by users.

Related requirements and use cases: FR-38, FR-39; UC-08 - UC-10

3.5 Non-Functional Requirements

The following quality requirements apply to the whole system and are stated in measurable terms. "Normal load" means up to 1,000 concurrent users.

3.5.1 Performance

ID	Requirement
NFR-P-01	The system shall support at least 1,000 concurrent users while meeting NFR-P-02 through NFR-P-04.
NFR-P-02	95% of browse and search pages shall load within 2 seconds under normal load.
NFR-P-03	95% of seat-selection, hold, and checkout steps shall complete within 3 seconds, excluding the payment service's response time.
NFR-P-04	A change in ticket or seat availability shall appear on every sales channel within 2 seconds.
NFR-P-05	99% of ticket validations (scan to result) shall complete within 1 second.

3.5.2 Reliability

ID	Requirement
NFR-R-01	Each approved payment shall produce exactly one transaction, even if the customer submits the order more than once.
NFR-R-02	No assigned seat shall ever be sold to more than one customer, verified by a test in which 500 simultaneous requests target the same seat.

ID	Requirement
NFR-R-03	Inventory from expired holds and failed payments shall be released within 10 seconds of the hold's expiration.
NFR-R-04	The database shall be backed up automatically at least every 24 hours, and backups shall be kept for at least 30 days.
NFR-R-05	After a system failure, no confirmed purchase or refund shall be lost, and at most 15 minutes of other data may need to be re-entered.

3.5.3 Availability

ID	Requirement
NFR-A-01	The customer ticketing service shall be available at least 99.9% of each calendar month (no more than about 43 minutes of unplanned downtime), excluding scheduled maintenance.
NFR-A-02	Scheduled maintenance shall take place only between 2:00 a.m. and 6:00 a.m. Pacific Time.
NFR-A-03	Customers and employees shall receive at least 24 hours' notice of scheduled maintenance.
NFR-A-04	If any one server fails it shall not interrupt ticket sales for more than 5 minutes.
NFR-A-05	An outage of the movie review service or email service shall not stop browsing, purchasing, or validation.

3.5.4 Security

ID	Requirement
NFR-S-01	All data in transit shall be encrypted with TLS 1.2 or higher; personal data at rest shall be encrypted.
NFR-S-02	Passwords shall be stored only as salted hashes (e.g., using bcrypt), never as plain text.
NFR-S-03	The system shall never store full card numbers or security codes; card handling shall comply with PCI DSS through the payment service's tokenization.
NFR-S-04	Every employee function shall require sign-in and role-based permission (Box Office, Administrator, Manager).
NFR-S-05	An employee account shall be locked for 15 minutes after 5 failed sign-in attempts, and all employee sign-in attempts shall be logged.
NFR-S-06	Ticket validation codes shall be randomly generated values at least 128 bits long so they cannot be guessed or forged.
NFR-S-07	Kiosks shall erase all customer and payment information at the end of each session or after 60 seconds of inactivity.
NFR-S-08	The system shall challenge (e.g., with a CAPTCHA) any device or account making more than 10 checkout attempts per minute and block it after repeated failures.

3.5.5 Maintainability

ID	Requirement
NFR-M-01	Theaters, auditoriums, showtimes, prices, and sales-window settings shall be changeable through the administrator interface without changing source code.
NFR-M-02	Every logged error shall include a timestamp, component, severity, request or transaction ID, and message; error logs shall be kept for 90 days.
NFR-M-03	Automated tests shall cover at least 80% of the server code.
NFR-M-04	Software updates shall be installable without losing transaction, account, ticket, or configuration data.

3.5.6 Portability

ID	Requirement
NFR-PT-01	The customer and employee interfaces shall work in the current and previous major versions of Chrome, Safari, Firefox, and Edge.
NFR-PT-02	Customers shall not need to install any application or plug-in.
NFR-PT-03	Layouts shall adapt to screen widths from 360 pixels (phones) to 1920 pixels (desktops and kiosks).
NFR-PT-04	Employee workstations shall be able to run the system on Windows, macOS, or ChromeOS.

3.5.7 Usability and Accessibility

ID	Requirement
NFR-U-01	In usability testing, at least 90% of first-time participants shall complete a standard purchase without help.
NFR-U-02	The median time to complete a standard purchase in usability testing shall not exceed 3 minutes.
NFR-U-03	Customer-facing pages shall meet WCAG 2.1 Level AA.

4. Software Design Specification

This section describes how the CMC Ticketing System would be built. It is written for the developers who would implement and maintain the system, and it covers a short description of the system (4.1), the software architecture (4.2), the UML class diagram (4.3), the classes with their attributes and operations (4.4), and the development plan (4.5).

4.1 System Description

The CMC Ticketing System is a web-based ticketing system shared by all 20 Chinese Multi-Cinema theaters in San Diego. Customers use the website or a lobby kiosk to browse showtimes, buy up to 20 tickets, choose seats in deluxe auditoriums, and receive tickets by email or print, either as guests or with an optional account. Employees use the same system to sell, scan and refund tickets; administrators manage movies, showtimes and prices; and managers view sales reports.

The design is object-oriented and uses a three-tier client-server architecture. All three sales channels are web browser clients that talk to one central server and one central database, so every theater, kiosk and box office sees the same showtimes, prices and seat availability (SI-03). Section 6 describes how that database is built, copied, and protected. The classes in Section 4.4 build on the conceptual classes in Section 3.4.

4.2 Software Architecture Overview

Figure 2 shows the major components of the system (solid boxes), grouped into tiers (dashed boxes), and the connectors between them (arrows K1 to K5).

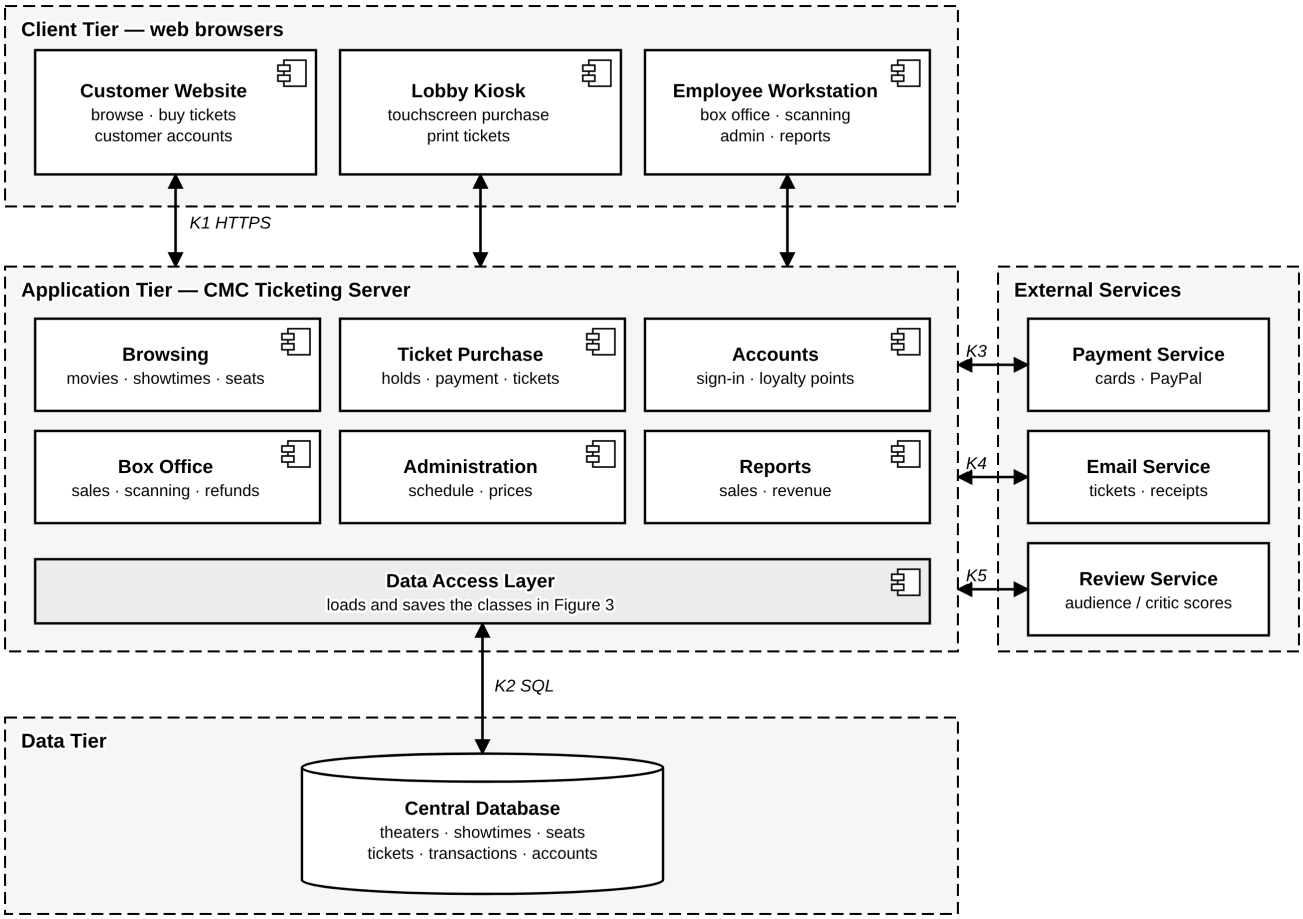


Figure 2. Software architecture diagram of the CMC Ticketing System

Note (Version 4.0): Figure 2 is the original Version 2.0 architecture. It is updated by Figure 5 in Section 6.1, which shows the data tier in detail.

Components. Table 4-1 describes each component in Figure 2.

Table 4-1. *Components*

Component	Tier	Description
Customer Website	Client	Browser application for customers on phones and computers: browse showtimes, buy tickets, and manage an optional account.
Lobby Kiosk	Client	Touchscreen version of the purchase flow in each theater lobby; prints tickets.
Employee Workstation	Client	Browser application for staff. It shows box-office, scanning, administration or report screens depending on the employee's role.
Browsing	Server module	Returns movies, showtimes, prices, seats remaining and review scores (FR-01 to FR-06).
Ticket Purchase	Server module	Holds tickets for five minutes, takes payment and issues tickets (FR-07 to FR-21).
Accounts	Server module	Sign-in, customer accounts, loyalty points and membership (FR-22 to FR-28).
Box Office	Server module	Walk-up sales, ticket scanning and refunds (FR-29 to FR-33).
Administration	Server module	Movies, showtimes, auditoriums, prices and transaction overrides (FR-34 to FR-40).

Component	Tier	Description
Reports	Server module	Sales and revenue reports for CMC management (FR-41 to FR-43).
Data Access Layer	Server	Loads and saves the objects of Figure 3 to and from the database for every module.
Central Database	Data	Stores theaters, showtimes, seat status, tickets, transactions and accounts.
Payment, Email and Review Services	External	Outside services that approve payments and refunds, send emails, and supply review scores.

Connectors. Table 4-2 describes how the components communicate.

Table 4-2. *Connectors*

ID	Connects	Protocol	Purpose
K1	Clients and server	HTTPS	Carries every request and response, encrypted with TLS (CI-01).
K2	Data Access Layer and database	SQL	Reads and saves data. Seat changes are saved one at a time so a seat cannot be sold twice (FR-12).
K3	Server and Payment Service	HTTPS API	Approves payments and refunds. CMC stores only the returned payment token (SI-01).
K4	Server and Email Service	HTTPS API	Sends confirmations and digital tickets (SI-04).
K5	Server and Review Service	HTTPS API	Gets review scores. If the service does not answer, showtimes are shown without scores (SI-02).

Example. When a customer buys tickets on the website, the request travels over K1 to the Ticket Purchase module. The module holds the selected seats through the Data Access Layer (K2), sends the payment to the Payment Service (K3), saves the transaction and tickets (K2), and has the Email Service send the tickets to the customer (K4). Because every channel uses the same database, those seats immediately show as sold on the kiosks and at the box office.

4.3 UML Class Diagram

Figure 3 shows the system's classes and their attributes, operations and relationships. The design keeps the ten classes from Section 3.4 and adds two more: an abstract Account class that holds the sign-in data customers and employees share, and a PaymentGateway interface for the external payment service.

Notation. A minus sign (-) marks a private attribute and a plus sign (+) a public operation. An italic class name is abstract. A hollow triangle shows inheritance, a filled diamond shows composition (the part belongs to the whole), a solid arrow shows an association with its multiplicity at each end, and a dashed arrow shows that one class uses another.

Relationships. CustomerAccount and EmployeeAccount inherit from the abstract Account class. A Theater contains one or more Auditoriums. Each Showtime shows one Movie in one Auditorium. A TicketHold reserves seats in one Showtime, and each Ticket admits one person to one Showtime. A Transaction includes 1 to 20 Tickets; it uses a TicketHold to issue those tickets and the PaymentGateway to take payment. A CustomerAccount can place any number of Transactions, and an EmployeeAccount creates an AuditRecord for each sensitive action.

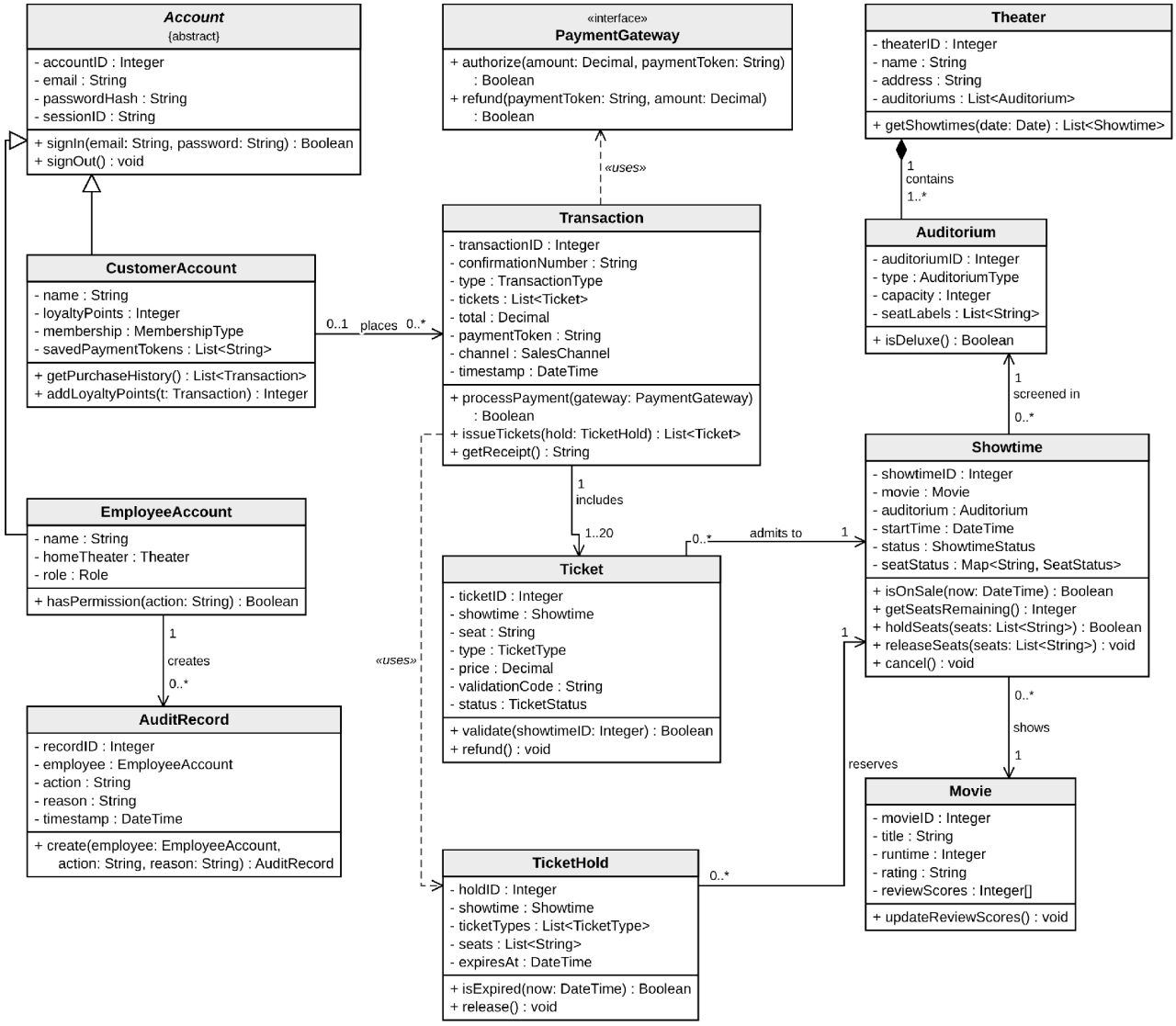


Figure 3. UML class diagram of the CMC Ticketing System

4.4 Class Descriptions

Each class is described below. Data types match those detailed in Section 3.4: Integer, String, Decimal (money), DateTime, Boolean, and List or Map for collections. Some attributes use enumerated types (Enums), either directly or as part of a collection. Each Enum is restricted to a predefined set of options listed in the corresponding descriptions. Operations show their parameters in parentheses and their return type in the second column.

4.4.1 Theater

One of the 20 CMC theater locations. Each theater contains its own auditoriums.

Attribute / Operation	Type / Returns	Description
- theaterID	Integer	Unique ID
- name	String	Theater name shown to customers
- address	String	Street address

Attribute / Operation	Type / Returns	Description
- auditoriums	`List<Auditorium>`	The theater's screening rooms
+ getShowtimes(date: Date)	`List<Showtime>`	Returns the showtimes at this theater on the given date (FR-01).

4.4.2 Auditorium

A screening room in a theater. Regular rooms sell 150 general-admission tickets, and deluxe rooms have 75 assigned seats.

Attribute / Operation	Type / Returns	Description
- auditoriumID	Integer	Unique ID
- type	AuditoriumType	REGULAR or DELUXE
- capacity	Integer	150 for regular, 75 for deluxe
- seatLabels	`List<String>`	Seat names such as "F12" (deluxe only)
+ isDeluxe()	Boolean	True if seats are assigned (FR-09).

4.4.3 Movie

A film that CMC is showing, with review scores from the licensed review service.

Attribute / Operation	Type / Returns	Description
- movieID	Integer	Unique ID
- title	String	Movie title
- runtime	Integer	Length in minutes
- rating	String	MPAA rating, e.g., PG-13
- reviewScores	Integer[]	Audience and critic scores (FR-06)
+ updateReviewScores()	void	Gets the latest scores from the review service; keeps the old scores if it does not respond.

4.4.4 Showtime

One screening of a movie in an auditorium. It tracks the status of every seat, so all holds and sales go through it (FR-12).

Attribute / Operation	Type / Returns	Description
- showtimeID	Integer	Unique ID
- movie	Movie	The movie being shown
- auditorium	Auditorium	Auditorium where the movie is shown
- startTime	DateTime	Scheduled start time
- status	ShowtimeStatus	SCHEDULED, ON_SALE, CLOSED or CANCELLED
- seatStatus	`Map<String,SeatStatus>`	AVAILABLE, HELD or SOLD for each seat

Attribute / Operation	Type / Returns	Description
+ isOnSale(now: DateTime)	Boolean	True from exactly 14 days before the start up to, but not including, 10 minutes after the start (FR-04, FR-05).
+ getSeatsRemaining()	Integer	Number of unsold seats (FR-02)
+ holdSeats(seats: `List<String>`)	Boolean	Marks the seats HELD; returns false if any is already held or sold.
+ releaseSeats(seats: `List<String>`)	void	Makes held or refunded seats available again (FR-13).
+ cancel()	void	Cancels the showtime and flags its tickets for refund (FR-40).

4.4.5 TicketHold

A five-minute reservation created when checkout begins (FR-11).

Attribute / Operation	Type / Returns	Description
- holdID	Integer	Unique ID
- showtime	Showtime	The showtime being held
- ticketTypes	`List<TicketType>`	Ticket type for each ticket (at most 20, FR-08)
- seats	`List<String>`	Held seats (deluxe only)
- expiresAt	DateTime	Creation time plus 5 minutes
+ isExpired(now: DateTime)	Boolean	True after expiresAt
+ release()	void	Returns the held seats to the showtime (FR-13).

4.4.6 Ticket

Admission for one person to one showtime, with a unique code that can be scanned once.

Attribute / Operation	Type / Returns	Description
- ticketID	Integer	Unique ID
- showtime	Showtime	Showtime associated with the ticket
- seat	String	Assigned seat, or empty for general admission
- type	TicketType	ADULT, CHILD, STUDENT, SENIOR or MILITARY
- price	Decimal	Price paid
- validationCode	String	Random code printed as a QR code (FR-20)
- status	TicketStatus	ISSUED, REDEEMED or REFUNDED
+ validate(showtimeID: Integer)	Boolean	Redeems the ticket on its first scan; rejects used, refunded or wrong-showtime tickets (FR-31, FR-32).
+ refund()	void	Marks the ticket refunded and frees its seat (FR-33).

4.4.7 Transaction

A purchase, refund or override. A purchase is recorded only after the payment is approved (FR-18, FR-19).

Attribute / Operation	Type / Returns	Description
- transactionID	Integer	Unique ID
- confirmationNumber	String	Number shown to the customer
- type	TransactionType	PURCHASE, REFUND or OVERRIDE
- tickets	`List<Ticket>`	Tickets in the transaction (1 to 20)
- total	Decimal	Amount charged or refunded
- paymentToken	String	Payment-service token; no card numbers are stored
- channel	SalesChannel	WEB, KIOSK or BOX_OFFICE
- timestamp	DateTime	When it was recorded
+ processPayment(gateway: PaymentGateway)	Boolean	Sends the total to the payment service; true if approved.
+ issueTickets(hold: TicketHold)	`List<Ticket>`	Creates the tickets and marks the held seats sold.
+ getReceipt()	String	Returns the receipt for the screen, email or printer.

4.4.8 Account

Abstract class. The parent class for anyone who signs in. It holds the sign-in data that customers and employees share, and only one session per account is allowed (FR-27).

Attribute / Operation	Type / Returns	Description
- accountID	Integer	Unique ID
- email	String	Sign-in name
- passwordHash	String	Hashed password, never plain text (NFR-S-02)
- sessionID	String	Current session; signing in elsewhere replaces it.
+ signIn(email: String, password: String)	Boolean	Checks the password and starts a new session.
+ signOut()	void	Ends the session

4.4.9 CustomerAccount

An optional account for returning customers (FR-23 to FR-28). Inherits from Account.

Attribute / Operation	Type / Returns	Description
- name	String	Customer's name
- loyaltyPoints	Integer	Current point balance
- membership	MembershipType	NONE or MEMBER
- savedPaymentTokens	`List<String>`	Saved payment methods, stored as tokens only
+ getPurchaseHistory()	`List<Transaction>`	Past and upcoming purchases (FR-24)
+ addLoyaltyPoints(t: Transaction)	Integer	Adds points for a purchase and returns the new balance (FR-25).

4.4.10 EmployeeAccount

A CMC staff account. Its role decides which employee functions it can use (NFR-S-04). Inherits from Account.

Attribute / Operation	Type / Returns	Description
- name	String	Employee's name
- homeTheater	Theater	Theater where the employee works
- role	Role	BOX_OFFICE, ADMINISTRATOR or MANAGER
+ hasPermission(action: String)	Boolean	Checks the role before any employee function.

4.4.11 AuditRecord

A permanent record of a refund, override, or price or schedule change (FR-39).

Attribute / Operation	Type / Returns	Description
- recordID	Integer	Unique ID
- employee	EmployeeAccount	Employee who performed the action
- action	String	Action that was performed
- reason	String	Reason entered by the employee
- timestamp	DateTime	Date and time when the record was created
+ create(employee: EmployeeAccount, action: String, reason: String)	AuditRecord	Creates a record; records cannot be edited or deleted.

4.4.12 PaymentGateway

Interface. Connects the system to the outside payment service (SI-01). It has no attributes; keeping it as an interface lets CMC change payment providers without changing other classes.

Attribute / Operation	Type / Returns	Description
+ authorize(amount: Decimal, paymentToken: String)	Boolean	Requests a charge; true if approved.
+ refund(paymentToken: String, amount: Decimal)	Boolean	Requests a refund; true if approved.

4.5 Development Plan and Timeline

We will develop the system in eight tasks between October and early December 2026. We will plan the data model during setup and build the database alongside the server modules from Figure 2, followed by the user interface, testing, and integration.

4.5.1 Partitioning of Tasks

#	Task	Description	Lead	Support	Target date
1	Project setup and data planning	Set up the repository and coding standards; outline the data model from the classes in Section 4.4.	Joshua H.	All	16 Oct 2026
2	Browsing module	Theater, Auditorium, Movie and Showtime classes; showtime search, seats remaining and review scores.	Matthew W.	Luke F.	23 Oct 2026

#	Task	Description	Lead	Support	Target date
3	Ticket Purchase module	TicketHold, Ticket, Transaction and PaymentGateway; five-minute holds, payment and ticket emails.	Luke F.	Joshua H.	6 Nov 2026
4	Accounts module	Account, CustomerAccount and EmployeeAccount; sign-in, loyalty points and membership.	Joshua H.	Matthew W.	6 Nov 2026
5	Box Office and Administration modules	Ticket scanning and refunds; managing schedules and prices; AuditRecord.	Joshua H.	Luke F.	20 Nov 2026
6	User interface and reports	Customer website, lobby kiosk and employee screens; sales and revenue reports.	Matthew W.	Joshua H.	20 Nov 2026
7	Testing	Write test cases from the functional requirements and use cases; fix defects found.	Luke F.	All	4 Dec 2026
8	System integration and deployment	Integrate the modules, deploy to all 20 theaters and verify shared seat inventory.	All	-	11 Dec 2026

4.5.2 Team Member Responsibilities

Member	Role	Tasks led	Responsibilities
Luke F.	Back-end developer (sales)	3, 7	Ticket holds, payment and ticket issuing; leads testing and tracks defects.
Matthew W.	Front-end developer	2, 6	Browsing module, customer website, kiosk and employee screens, and reports.
Joshua H.	Database and security developer	1, 4, 5	Repository setup and data planning, database work alongside the modules, accounts and sign-in, box office and administration functions, audit records.

4.6 Design Updates Since Version 2.0

We reviewed the architecture (Figure 2) and the class diagram (Figure 3) against the test plan in Section 5. **No classes, attributes, operations, or relationships were added or removed**, so Figures 2 and 3 are unchanged in this version. Every component and class that the test plan targets already exists in the design, and several existing design choices are what make the planned tests possible:

- **PaymentGateway is an interface (4.4.12).** Unit and integration tests replace the real payment service with a test version (*FakePaymentGateway*) that approves, declines, or times out on demand, so payment failures can be tested without real charges.
- **Time is passed in as a parameter.** `Showtime.isOnSale(now)` and `TicketHold.isExpired(now)` take the current time as an argument, so tests can check the 14-day, 10-minute, and 5-minute boundaries exactly, without waiting.
- **Showtime is the single owner of seat status (4.4.4).** Because every hold and sale goes through one class and one connector (K2), the double-sale risk can be tested in one place at every level (TS-U1, TS-I1, TS-S1).
- **AuditRecord is write-once (4.4.11).** Tests can confirm that an audit entry exists and cannot be changed afterward.

Two changes were made while writing the test vectors:

1. **Clarified sales-window boundary.** The description of `Showtime.isOnSale` in 4.4.4 now states that sales open *exactly* 14 days before the start time and close *at* 10 minutes after it (so a request at 7:09:59 PM for a 7:00

PM showing is accepted and one at 7:10:00 PM is refused). The old wording, "until 10 minutes after," did not say which side of the boundary was included, and a test needs a single correct answer.

2. **Added Figure 4.** Section 5.3 adds a test-scope view of the architecture. It is a copy of Figure 2 marked with the test sets that cover each component and connector; it does not change the design.

5. Test Plan

This section explains how the CMC Ticketing System will be tested. It covers both kinds of checking described in the course reading:

- **Verification** - are we building the product right? Each class, module, and connector is checked against the design in Section 4.
- **Validation** - are we building the right product? The finished system is checked against the user requirements in Section 2.2 and the measurable requirements in Section 3.5.

Every test set below names the design elements it targets (Section 4), the requirements it checks (Section 3), and the failures it is meant to catch, so a failed test points both to the broken requirement and to the place in the design to look.

GitHub repository: The test plan, this SRS, and the test case workbook are stored in the group repository at **`https://github.com/7mwang/CMC-Ticketing`** (folder `test-plan/`).

5.1 Test Objectives

The test plan has five objectives:

1. Show that each class in Section 4.4 behaves as its operation descriptions state, including exactly at the edges of CMC's business rules: the 14-day sales window, the 10-minute sales close, the 5-minute hold, and the 20-ticket limit.
2. Show that the modules work together across connectors K2-K5, including when an external service declines, fails, or does not answer.
3. Show that every Must-have functional requirement works through its use case, the way a customer or employee would use it.
4. Measure the system against the non-functional requirements in Section 3.5 (performance, reliability, availability, security, and usability) on a production-like installation.
5. Show that the two failures CMC cares about most cannot happen: **an assigned seat sold to two customers** (FR-12, NFR-R-02) and **a customer charged twice or given tickets without paying** (FR-18, NFR-R-01).

5.2 Test Strategy

5.2.1 Test Levels

Testing runs at four levels, from the smallest piece of code to the complete system. The plan covers the three standard granularities (unit, functional, and system) with at least two test sets each, and adds an integration level between unit and functional because the highest-risk behavior (holding seats and taking payment) happens where modules connect.

Table 5-1. *Test levels*

Level	What is tested	Approach	Run by	When (Section 4.5 task)	Test sets
Unit	One class and its operations, with every other class replaced by a simple stand-in (stub)	White-box: branch coverage and boundary values	The developer who wrote the class	Tasks 2-5, as each class is written	TS-U1, TS-U2, TS-U3
Integration	Two or more classes or modules working through a connector, using a real test database and stubbed external services	Interface testing: data passed between components, error returns, and saved data	The module leads	Tasks 3-5, after the classes pass their unit tests	TS-I1, TS-I2
Functional	One feature (use case) at a time, driven through the website, kiosk, or employee screens	Black-box: equivalence classes, boundary values, decision tables, and use-case scenarios	Test lead (Luke F.) with one other member	Task 7	TS-F1, TS-F2, TS-F3
System	The complete installed system, all three channels at once, measured against Section 3.5	Black-box: load, stress, concurrency, security, failover, and usability testing	All members	Tasks 7-8	TS-S1, TS-S2, TS-S3, TS-S4

Regression testing. All automated unit, integration, and functional tests are re-run after every change merged into the repository, so a fix in one module cannot silently break another. The Reports module (FR-41 to FR-43) is a read-only module with low risk and is covered only by this regression suite in the first release

5.2.2 Test Design Techniques

The test vectors in Sections 5.4-5.7 were chosen with the following techniques from the course reading, so that a small number of tests covers each feature thoroughly:

- **Equivalence classes.** Inputs are split into groups that the system should treat the same way, and one value is tested from each group. For example, ticket quantity has three classes: 0 or fewer (invalid), 1-20 (valid), and 21 or more (invalid).
- **Boundary values.** Most defects occur at limits, so each limit is tested on the boundary and just on each side of it: 20 and 21 tickets; a hold at 4:59, 5:00, and 5:01; sales opening exactly 14 days before the show and closing exactly 10 minutes after it starts.
- **State-based testing.** Tickets, seats, and showtimes move between fixed states (for example, a seat is AVAILABLE, HELD, or SOLD). Every allowed change and every forbidden change is tested once (TS-U2).
- **Decision tables (cause and effect).** When an outcome depends on a combination of conditions, every combination is listed as a rule and tested. Refund eligibility uses this technique (TS-F2).
- **Control-flow (branch) coverage.** Unit tests run every branch of the targeted operations in both directions (true and false) at least once.

5.2.3 Test Environment and Test Data

- **Environment.** A test installation that matches production: one server, one database, and the three client types: the four supported browsers (current and previous versions, NFR-PT-01), a touchscreen kiosk, and an employee workstation with a ticket printer and scanner.
- **Test clock.** The server's clock can be set by the tester, so time-based rules can be tested in seconds instead of days.

- **Stand-ins for external services.** The payment service is replaced by *FakePaymentGateway* (an implementation of the PaymentGateway interface) in unit and integration tests, and by the payment processor's own sandbox in functional and system tests. The email and review services use a sandbox and a stub, and can be switched off to test the "service unavailable" flows.
- **Test data.** Two theaters. Theater 1 has Auditorium 1 (regular, 150 general-admission seats) and Auditorium 2 (deluxe, 75 seats in rows A-E, numbered 1-15). Two showtimes of a 120-minute test movie start at **7:00 PM on Friday, 20 November 2026**: **SHOW-REG** in Auditorium 1 and **SHOW-DLX** in Auditorium 2. Test prices are ADULT $15.00 and CHILD, STUDENT, SENIOR, and MILITARY $11.00, with a $1.00 fee per ticket on the website and kiosk. Test accounts exist for each role: E100 (BOX_OFFICE), E200 (ADMINISTRATOR), and E300 (MANAGER), plus one customer account.
- **Tools.** An automated unit-testing framework and code-coverage tool (coverage target of 80%, NFR-M-03); a load-testing tool such as Apache JMeter for TS-S1 and TS-S2; an automated accessibility checker for TS-S4; and GitHub Issues in the repository for tracking defects.

5.2.4 Entry and Exit Criteria

Entry criteria. A test level starts only when the level below it has passed: integration tests run on classes that pass their unit tests, functional tests run on modules that pass integration, and system tests run on the fully integrated build. So, tests are written against fixed requirement IDs.

Defect severity.

Severity	Meaning	Example
Critical	Money, seats, or data are wrong, or security is broken	A seat is sold twice; a card is charged twice
High	A Must-have feature does not work and has no workaround	A valid ticket is rejected at the door
Medium	A feature works incorrectly but has a workaround	The refund confirmation does not print but can be emailed
Low	Cosmetic issue	A misaligned label on the seat map

Exit criteria. Testing is complete and the system can be deployed (Task 8) when:

- every P0 test case passes;
- no Critical or High defects are open;
- automated tests cover at least 80% of the server code;
- every Must-have requirement is covered by at least one passing test (Table 5-7); and
- every measurable requirement in Section 3.5 is met in TS-S1 to TS-S4.

5.3 Test Scope on the Design

Figure 4 places the test plan on the software architecture from Figure 2. Each component and connector is marked with the test sets that cover it. Unit tests (blue) target classes inside the server modules; integration tests (purple) target the connectors between modules, the database, and the payment service; functional tests (green) enter through the three client applications; and system tests (orange) cover everything inside the dashed orange boundary. The external services sit outside the boundary because CMC does not own them: we test how our system handles their answers, not the services themselves. The Ticket Purchase module is outlined in red because the two most serious failures in Section 5.1 would happen there.

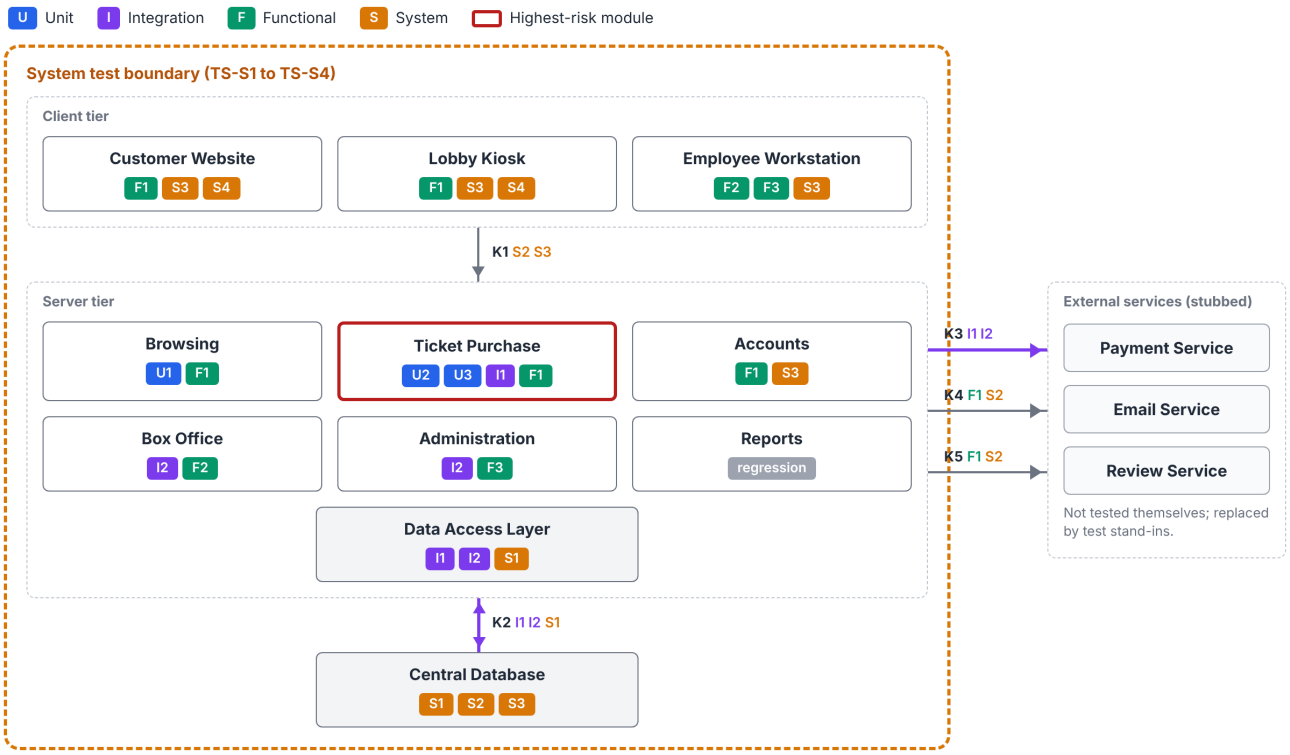


Figure 4. Test scope of the CMC Ticketing System, drawn on the architecture of Figure 2

Table 5-2. *Design elements, test sets, and failures covered*

Design element (Section 4)	Test sets	Failures the tests are designed to catch
Showtime (4.4.4)	TS-U1, TS-I1, TS-S1	Sales open or close one minute early or late; a held or sold seat can be held again; a partial hold leaves seats stuck in HELD
TicketHold (4.4.5)	TS-U3, TS-F1	A hold expires early or never expires; more than 20 tickets are accepted; the number of seats does not match the number of tickets
Ticket (4.4.6)	TS-U2, TS-F2	A ticket admits two people; a refunded ticket or a ticket for another showtime is admitted; validation codes repeat
Transaction + PaymentGateway (4.4.7, 4.4.12, K3)	TS-I1, TS-F1, TS-S1	Tickets issued without approved payment; a double submission charges twice; an approved payment is not saved
Box Office module + AuditRecord (4.4.11)	TS-I2, TS-F2	A refund after the show starts; a rejected refund still cancels the ticket; a refund without an audit entry
Administration module	TS-I2, TS-F3	Overlapping showtimes; negative or missing prices; a change that does not reach the other channels
Account, EmployeeAccount (4.4.8-4.4.10)	TS-F1, TS-S3	An employee function reachable without the right role; unlimited password guessing; two active sessions
Data Access Layer + Central Database (K2)	TS-I1, TS-I2, TS-S1	Two simultaneous requests both succeed in buying one seat; saved data does not match the objects
K1 (HTTPS)	TS-S2, TS-S3	Slow responses under load; unencrypted or outdated-TLS connections accepted
K4 and K5 (email, reviews)	TS-F1, TS-S2	An email or review service outage stops browsing or purchasing
Client applications	TS-F1 to TS-F3, TS-S4	Purchase flow too slow or confusing; kiosk keeps personal data; pages fail accessibility checks

Not tested in this release. Features outside the scope in Section 1.2 (a native mobile app, online self-service refunds, concessions) are not tested. Could-have requirements (FR-17 Bitcoin, FR-28 membership purchase, FR-43 CSV export, FR-44 satisfaction rating).

5.4 Unit Test Sets

Each unit test set exercises one class in isolation. Classes that the target class depends on are replaced by stubs, and the test clock supplies the current time.

5.4.1 TS-U1: Showtime Sales Window and Seat Status

Target. The Showtime class (4.4.4): `isOnSale(now)`, `holdSeats(seats)`, `releaseSeats(seats)`, and `getSeatsRemaining()`.

Requirements. FR-02, FR-04, FR-05, FR-12, FR-13.

Why this class. Every channel holds and sells seats through Showtime, so a defect here becomes a defect on the website, the kiosk, and the box office at the same time.

Method. Boundary values for the sales window; state-based tests for seat status; 100% branch coverage of the four operations.

Table 5-3. *TS-U1 test vectors (SHOW-DLX starts 20 Nov 2026, 7:00 PM)*

Vector	Input	Expected result
U1-1	`isOnSale(6Nov, 6:59PM)` - 1 minute before sales open	false
U1-2	`isOnSale(6Nov, 7:00PM)` - exactly 14 days before	true
U1-3	`isOnSale(20Nov, 7:09:59PM)` - just before sales close	true
U1-4	`isOnSale(20Nov, 7:10:00PM)` - exactly 10 minutes after start	false
U1-5	`isOnSale(15Nov, noon)` with status CANCELLED	false
U1-6	`holdSeats(["C7", "C8"])`, both AVAILABLE	true; C7 and C8 become HELD
U1-7	`holdSeats(["C8", "C9"])` after U1-6 (C8 already HELD)	false; C9 stays AVAILABLE (the hold is all-or-nothing)
U1-8	`holdSeats(["Z99"])` - a seat that does not exist	false; no seat changes
U1-9	`releaseSeats(["C7", "C8"])`, then `getSeatsRemaining()`	C7 and C8 AVAILABLE; 75

Pass criteria. All nine vectors give the expected result and every branch of the four operations is executed.

5.4.2 TS-U2: Ticket Validation and Refund States

Target. The Ticket class (4.4.6): `validate(showtimeID)` and `refund()`.

Requirements. FR-20, FR-31, FR-32, FR-33, NFR-S-06.

Method. State-based testing. A ticket can be ISSUED, REDEEMED, or REFUNDED. Table 5-4 lists every combination of starting state and action; each cell is one test vector, so the set tests every allowed and every forbidden change of state.

Table 5-4. *TS-U2 state transition vectors*

Starting state	`validate()` for the correct showtime	`validate()` for a different showtime	`refund()`
ISSUED	true; becomes REDEEMED	false ("wrong showtime"); stays ISSUED	becomes REFUNDED; seat returned to Showtime
REDEEMED	false ("already used"); stays REDEEMED	false; stays REDEEMED	refused; stays REDEEMED
REFUNDED	false ("refunded"); stays REFUNDED	false; stays REFUNDED	refused; stays REFUNDED

A tenth vector generates 100,000 tickets and checks that no two validation codes are the same and that each code carries at least 128 random bits (NFR-S-06).

Pass criteria. All ten vectors give the expected result.

5.4.3 TS-U3: TicketHold Expiry and Ticket Limits

Target. The TicketHold class (4.4.5): creating a hold, `isExpired(now)`, and `release()`.

Requirements. FR-08, FR-09, FR-11, FR-13.

Method. Equivalence classes and boundary values for quantity; boundary values for expiry time.

Table 5-5. *TS-U3 test vectors (hold created at 7:00:00 PM, so expiresAt = 7:05:00 PM)*

Vector	Input	Expected result
U3-1	`isExpired(7:04:59PM)`	false
U3-2	`isExpired(7:05:00PM)`	false (the hold expires *after* expiresAt)
U3-3	`isExpired(7:05:01PM)`	true
U3-4	Create a hold with 0 tickets	rejected
U3-5	Create a hold with 1 ticket	accepted
U3-6	Create a hold with 20 tickets	accepted
U3-7	Create a hold with 21 tickets	rejected (FR-08)
U3-8	Deluxe hold with 3 tickets but only 2 seats	rejected (one seat per ticket, FR-09)
U3-9	`release()` called twice on the same hold	seats AVAILABLE after the first call; the second call changes nothing

Pass criteria. All nine vectors give the expected result.

5.5 Integration Test Sets

Integration tests combine classes that already passed their unit tests (bottom-up integration). They use a real test database through the Data Access Layer (K2) and FakePaymentGateway for the payment service (K3), and they check what is actually saved in the database, not only what an operation returns.

5.5.1 TS-I1: Purchase Pipeline

Target. Ticket Purchase module: TicketHold, Showtime, Transaction (`processPayment`, `issueTickets`), and PaymentGateway, across K2 and K3.

Requirements. FR-11, FR-13, FR-18, FR-19, FR-20, NFR-R-01, NFR-R-03.

Failures covered. Tickets issued without approved payment; a double charge; an approved payment that is not saved; held seats that are never released.

Vector	Setup and action	Expected result
I1-1	Hold D7 and D8 (2 ADULT, total $32.00); gateway set to APPROVE; submit payment	One APPROVED PURCHASE transaction with a confirmation number; two ISSUED tickets with different codes; D7 and D8 saved as SOLD
I1-2	Same as I1-1, but the order is submitted twice within one second	The gateway receives exactly one charge; exactly one transaction is saved
I1-3	Gateway set to DECLINE	No tickets issued; D7 and D8 stay HELD until the hold expires
I1-4	Gateway set to TIMEOUT (no answer)	No tickets issued; the hold stays active; the customer is told the payment status is being checked
I1-5	No payment; test clock moved to 5 minutes 1 second after the hold	D7 and D8 saved as AVAILABLE within 10 seconds

5.5.2 TS-I2: Refund and Audit Path

Target. Box Office module, Ticket (`refund`), PaymentGateway (`refund`), Showtime (`releaseSeats`), and AuditRecord, across K2 and K3.

Requirements. FR-33, FR-39, NFR-S-04, and the error handling in 3.2.5.5.

Failures covered. A rejected refund that still cancels the ticket; a refund without an audit entry; a refund by an employee without permission.

Vector	Setup and action	Expected result
I2-1	E100 refunds D7 of a two-ticket purchase before the show, reason "schedule conflict"; gateway APPROVES	D7 REFUNDED and AVAILABLE; D8 unchanged; one REFUND transaction; one AuditRecord with E100, action, reason, and timestamp
I2-2	Same, but the gateway REJECTS the refund	D7 stays ISSUED and SOLD; no REFUND transaction; the rejection reason is returned to the screen
I2-3	Same as I2-1 with an empty reason	Refused before the gateway is called
I2-4	E300 (MANAGER role, no refund permission) attempts I2-1	`hasPermission` returns false; refused; the attempt is logged
I2-5	After I2-1, an attempt is made to edit the AuditRecord	Refused; the record is unchanged

5.6 Functional Test Sets

Functional tests treat the system as a black box. A tester follows each use case from Section 3.3 through the real screens, using the processor's payment sandbox, and compares what happens with the use case's main scenario and its alternate flows.

5.6.1 TS-F1: Purchase Tickets (UC-01, UC-02, UC-03)

Target. Customer Website and Lobby Kiosk, through the Browsing, Ticket Purchase, and Accounts modules and the email and review services (K4, K5).

Requirements. FR-01 to FR-22, FR-45, UI-01 to UI-03, UI-05; user requirements UR-01 to UR-05.

Vector	Scenario	Expected result
F1-1	Guest on the website buys 2 ADULT for SHOW-DLX, seats D7 and D8, with an approved sandbox card (main scenario of UC-02 and UC-03)	Hold countdown shown; total $32.00 shown before payment; confirmation number displayed; tickets emailed within 2 minutes; D7 and D8 shown as sold on the kiosk
F1-2	Kiosk customer buys 2 ADULT and 2 CHILD for SHOW-REG	No seat map; total $56.00; finished in six screens or fewer; tickets print
F1-3	Customer selects 21 tickets, then 20 (UC-02 flow 2a)	21 is blocked with an explanation of the limit; 20 continues to checkout
F1-4	Two testers choose seat D9 at the same moment (UC-03 flow 3a)	One gets the seat; the other is told it is taken and asked to choose another
F1-5	Sandbox card declined, then PayPal used within the hold (flow 7a)	No tickets after the decline; the PayPal payment succeeds and tickets are issued once
F1-6	Customer waits more than 5 minutes on the payment page (flow 7b)	Hold-expired message; the seats are available again; the customer can restart
F1-7	Email and review services switched off (flows 9a and UC-01 4a)	Showtimes appear without scores; purchase completes; tickets can be printed from the confirmation screen
F1-8	Customer opens a showtime 15 days away (UC-01 flow 5a)	No buy button; the date when sales open is shown

5.6.2 TS-F2: Validate Tickets and Process Refunds (UC-07, UC-08)

Target. Employee Workstation, Box Office module, scanner and printer.

Requirements. FR-30 to FR-33, FR-39; UR-09, UR-10.

Ticket validation vectors. A tester scans (1) a newly issued ticket for the correct showtime, (2) the same ticket again, (3) a refunded ticket, (4) a ticket for a different showtime, and (5) types a random code. The expected results are "valid" with the auditorium and seat, then "already used," "refunded," "wrong showtime," and "not found." The tester also looks up one purchase by confirmation number, by ticket code, and by email address, and each search must find it (FR-30).

Refund vectors. Whether a refund is allowed depends on three conditions, so all combinations are listed in a decision table, following the method in the course reading. Each rule is one test vector.

Table 5-6. *Refund decision table*

	R1	R2	R3	R4	R5
Conditions					
Showtime has started	N	N	N	N	Y
Ticket status	ISSUED	ISSUED	REDEEMED	REFUNDED	any
Payment service approves the refund	Y	N	-	-	-
Actions					
Refund money and invalidate ticket	X				
Return seat to inventory and write audit record	X				
Refuse and show the reason		X	X	X	X
Ticket stays valid		X			

5.6.3 TS-F3: Manage Showtimes and Prices (UC-09)

Target. Employee Workstation (administrator screens) and the Administration module.

Requirements. FR-34 to FR-37, FR-39, FR-40, NFR-P-04, NFR-S-04. A showtime is treated as occupying its auditorium from its start time until start time plus the movie's runtime.

Vector	Action (administrator E200 unless noted)	Expected result
F3-1	Add a showtime in Auditorium 2 at 8:30 PM on 20 Nov (SHOW-DLX runs 7:00-9:00 PM)	Rejected as overlapping, with an explanation
F3-2	Add the same showtime at 9:00 PM	Accepted (boundary: starts exactly when the previous one ends)
F3-3	Add a 7:00 PM showtime in Auditorium 1 of Theater 2	Accepted (different auditorium)
F3-4	Set the ADULT price to -$1.00, then leave it blank, then $0.00	The first two are rejected; $0.00 is accepted (only negative or missing prices are invalid)
F3-5	Change the ADULT price from $15.00 to $16.00	The new price appears on the kiosk within 2 seconds; an audit record is created
F3-6	Cancel a showtime that has 5 sold tickets	All 5 tickets are marked refund-eligible and listed for follow-up (FR-40)
F3-7	E100 (BOX_OFFICE) opens the administration page	Access denied; the attempt is logged

5.7 System Test Sets

System tests run on the complete installation with all three channels active at once. Most of them measure the non-functional requirements in Section 3.5, which can only be checked on the whole system. TS-S4 is the validation step: real users confirm that the system meets their needs.

5.7.1 TS-S1: Seat Integrity and Cross-Channel Consistency

Target. The whole purchase path, especially the Data Access Layer, the database, and K2.

Requirements. FR-10, FR-12, SI-03, NFR-R-02, NFR-P-04.

- **S1-1 (deluxe seat race).** The load tool sends 500 simultaneous requests for seat E10, split across the website, kiosks, and box office. *Expected:* exactly one request gets the hold, 499 are told the seat is unavailable, and exactly one ticket for E10 exists after payment. The test is repeated 10 times.
- **S1-2 (regular auditorium capacity).** 160 customers each try to buy one ticket for SHOW-REG (150 seats) at the same time. *Expected:* exactly 150 tickets are sold; the auditorium is never oversold.
- **S1-3 (cross-channel update).** 100 purchases are made at the box office while a second tester watches the website and kiosk. *Expected:* each sold seat appears as sold on the other channels within 2 seconds in all 100 trials.

5.7.2 TS-S2: Performance, Load, and Availability

Target. The whole system, especially K1 and the database.

Requirements. FR-45, NFR-P-01 to NFR-P-05, NFR-R-05, NFR-A-04, NFR-A-05.

- **S2-1 (normal load).** 1,000 simulated users for one hour: 70% browsing, 25% buying, and 5% employee scanning and selling. *Expected:* 95% of browse pages load within 2 seconds, 95% of checkout steps finish within 3 seconds (not counting the payment service), and 99% of ticket scans finish within 1 second.

- **S2-2 (stress).** The load is raised to 2,000 users. *Expected:* the waiting queue starts and shows each customer's position; the system does not crash; no completed purchase is lost.
- **S2-3 (server failure).** One server is stopped during S2-1. *Expected:* sales resume within 5 minutes and no confirmed purchase or refund is lost.
- **S2-4 (external outage).** The email and review services are switched off during S2-1. *Expected:* browsing, purchasing, and ticket validation continue normally.

5.7.3 TS-S3: Security

Target. Accounts module, employee and kiosk clients, K1, and the database.

Requirements. FR-27, UI-06, CI-01, NFR-S-01 to NFR-S-05, NFR-S-07, NFR-S-08.

- **S3-1 (account lockout).** Five wrong passwords on E100, then the correct one. *Expected:* the sixth attempt is refused for 15 minutes even though the password is correct; all six attempts are logged.
- **S3-2 (role permissions).** Each role (BOX_OFFICE, ADMINISTRATOR, MANAGER, and a customer account) tries every employee function. *Expected:* only the functions allowed for that role succeed; denied attempts are logged.
- **S3-3 (encryption).** Connections are attempted over plain HTTP and over TLS 1.0 and 1.1. *Expected:* all are refused or redirected to TLS 1.2 or higher.
- **S3-4 (stored data).** After TS-F1, the tester searches the database for card numbers and plain-text passwords. *Expected:* none are found; passwords are stored only as salted hashes.
- **S3-5 (kiosk privacy).** A customer leaves the kiosk in the middle of checkout. *Expected:* all personal and payment data is cleared after 60 seconds.
- **S3-6 (bots).** A script makes 11 checkout attempts in one minute from one device and keeps going. *Expected:* a CAPTCHA appears, then the device is blocked.
- **S3-7 (single session).** A customer signs in on a phone, then on a laptop. *Expected:* the phone session ends.

5.7.4 TS-S4: Usability and Accessibility (Acceptance)

Target. Customer Website and Lobby Kiosk.

Requirements. UI-04, UI-05, NFR-U-01 to NFR-U-03; validates UR-01 to UR-05.

- **S4-1 (alpha test).** 20 first-time participants of different ages, some of whom prefer Spanish, are asked to buy two tickets for a deluxe showing without help. *Expected:* at least 18 of 20 (90%) succeed without help, and the median time is 3 minutes or less.
- **S4-2 (accessibility).** An automated checker and a manual keyboard and screen-reader review of the purchase flow. *Expected:* no WCAG 2.1 Level AA failures.
- **S4-3 (kiosk and language).** Measure the kiosk's touch targets and count its screens; switch to Spanish on every page. *Expected:* targets of at least 1.5 cm $\times$ 1.5 cm; six screens or fewer; every page offers Spanish.
- **S4-4 (beta test).** Before all 20 theaters go live in Task 8, the system runs at one theater for one week. CMC staff and customers report problems, which are fixed or scheduled before the full deployment.

5.8 Test Case Samples

Ten sample test cases are recorded in the course's Excel test case template, saved in the repository as `test-plan/CMC_TestCases_Group14.xlsx`. They cover all four levels (three unit, two integration, three functional,

and two system cases), and each one is a fully specified version of a vector from the test sets above. Because the system has not been built yet, the *Actual Result* and *Status* columns are marked "Not run"; they will be filled in when the tests are executed in Task 7. The *Test Executed By* column names the member assigned to run each case.

ID	Level	Component	Summary	From vector
UT-01	Unit	Showtime	Sales open exactly 14 days before and close 10 minutes after the start time	U1-1 to U1-4
UT-02	Unit	Showtime	A seat hold is all-or-nothing and cannot take an already-held seat	U1-6 to U1-9
UT-03	Unit	Ticket	A ticket is redeemed once; reused, wrong-showtime, and refunded tickets are rejected	Table 5-4
IT-01	Integration	Ticket Purchase + DAL + PaymentGateway	An approved payment creates one transaction and SOLD seats; a double submission charges once	I1-1, I1-2
IT-02	Integration	Box Office + PaymentGateway + AuditRecord	A rejected refund leaves the ticket valid; an approved refund frees the seat and is audited	I2-1, I2-2
FT-01	Functional	Customer Website	A guest buys two deluxe tickets with seat selection	F1-1
FT-02	Functional	Customer Website	21 tickets are blocked and 20 are accepted	F1-3
FT-03	Functional	Employee Workstation	Refunds are refused after the show starts and for a redeemed ticket	R3, R5
ST-01	System	Whole system	500 simultaneous requests for one seat produce exactly one sale	S1-1
ST-02	System	Accounts / Employee Workstation	Five failed sign-ins lock an employee account for 15 minutes	S3-1

5.9 Requirements Traceability

Table 5-7 maps each group of requirements to the test sets that cover it. Every Must-have requirement is covered by at least one test set, and most are covered at more than one level.

Table 5-7. *Traceability from requirements to test sets*

Requirements	Feature	Test sets
FR-01 to FR-06, UI-04	Browse movies and showtimes	TS-U1, TS-F1, TS-S4
FR-07 to FR-14, UI-02, UI-03	Select and hold tickets	TS-U1, TS-U3, TS-I1, TS-F1, TS-S1
FR-15 to FR-21, UI-01, SI-01, SI-04, CI-03	Purchase and deliver tickets	TS-I1, TS-F1, TS-S2
FR-22 to FR-27	Customer accounts	TS-F1, TS-S3
FR-29 to FR-33	Box office sales, validation, refunds	TS-U2, TS-I2, TS-F2
FR-34 to FR-40	Administration	TS-I2, TS-F3
FR-41, FR-42	Reports	Regression suite
FR-45, FR-46	Demand protection	TS-S2, TS-S3
NFR-P-01 to NFR-P-05	Performance	TS-S1, TS-S2
NFR-R-01 to NFR-R-03, NFR-R-05	Reliability	TS-I1, TS-S1, TS-S2
NFR-A-04, NFR-A-05	Availability	TS-S2
NFR-S-01 to NFR-S-08, CI-01, UI-06	Security	TS-U2, TS-I2, TS-S3
NFR-M-03	Test coverage	Exit criteria (5.2.4)
NFR-U-01 to NFR-U-03, UI-05, NFR-PT-01, NFR-PT-03	Usability, accessibility, portability	TS-S4

Requirements that are checked by inspection or by operating procedures rather than by tests (NFR-R-04 backups, NFR-A-01 to NFR-A-03 monthly uptime and maintenance windows, NFR-M-01, NFR-M-02, NFR-M-04, NFR-PT-02, NFR-PT-04) are reviewed during Task 8 before deployment.

5.10 Test Schedule and Responsibilities

The test schedule follows the development plan in Section 4.5: unit tests are written alongside each module, integration tests follow as modules are finished, and functional and system testing take place in Tasks 7 and 8.

Test sets	Owner	Runs during	Target date
TS-U1	Matthew W.	Task 2 (Browsing module)	23 Oct 2026
TS-U2, TS-U3	Luke F.	Task 3 (Ticket Purchase module)	6 Nov 2026
TS-I1	Luke F., with Joshua H.	Tasks 3-5, once the Ticket Purchase classes pass their unit tests	13 Nov 2026
TS-I2	Joshua H.	Task 5 (Box Office and Administration)	20 Nov 2026
TS-F1 to TS-F3	Luke F. (lead), Matthew W.	Task 7 (Testing)	27 Nov 2026
TS-S1 to TS-S3	All members	Task 7 (Testing)	4 Dec 2026
TS-S4	All members, with CMC staff	Task 8 (alpha and beta testing before full deployment)	11 Dec 2026

Defects found at any level are recorded as GitHub Issues with their severity, the failing test case ID, and the requirement ID. A defect is closed only after its test case passes again and the full regression suite still passes.

The testing process will be fully logged including results and statuses for each case. After testing occurs, a summary will be created that details whether the exit criteria in 5.2.4 are met and the system is ready to deploy.

6. Design 2.0: Architecture and Data Management

This section updates the Software Design Specification in Section 4. Version 2.0 of the design showed a single "Central Database" but did not say what kind of database it was, how the data inside it was organized, or how that data would be protected and recovered. This section fills that gap. It presents an updated architecture diagram that shows every data store (6.1), lists what changed since the original design and why (6.2), describes the data management strategy (6.3), and explains our design decisions, the alternatives we considered, and the tradeoffs between them (6.4).

The classes and relationships in Figure 3 and Section 4.4 do not change. This section describes where and how their data is stored.

6.1 Updated Software Architecture

Figure 5 shows the updated architecture. The client tier, the six server modules, and the three external services are the same as in Figure 2. The changes are in the data tier, which now has five parts instead of one database, and in the two server components that work with it: the Data Access Layer and the Reports module. Shaded boxes are new or changed since Version 2.0.

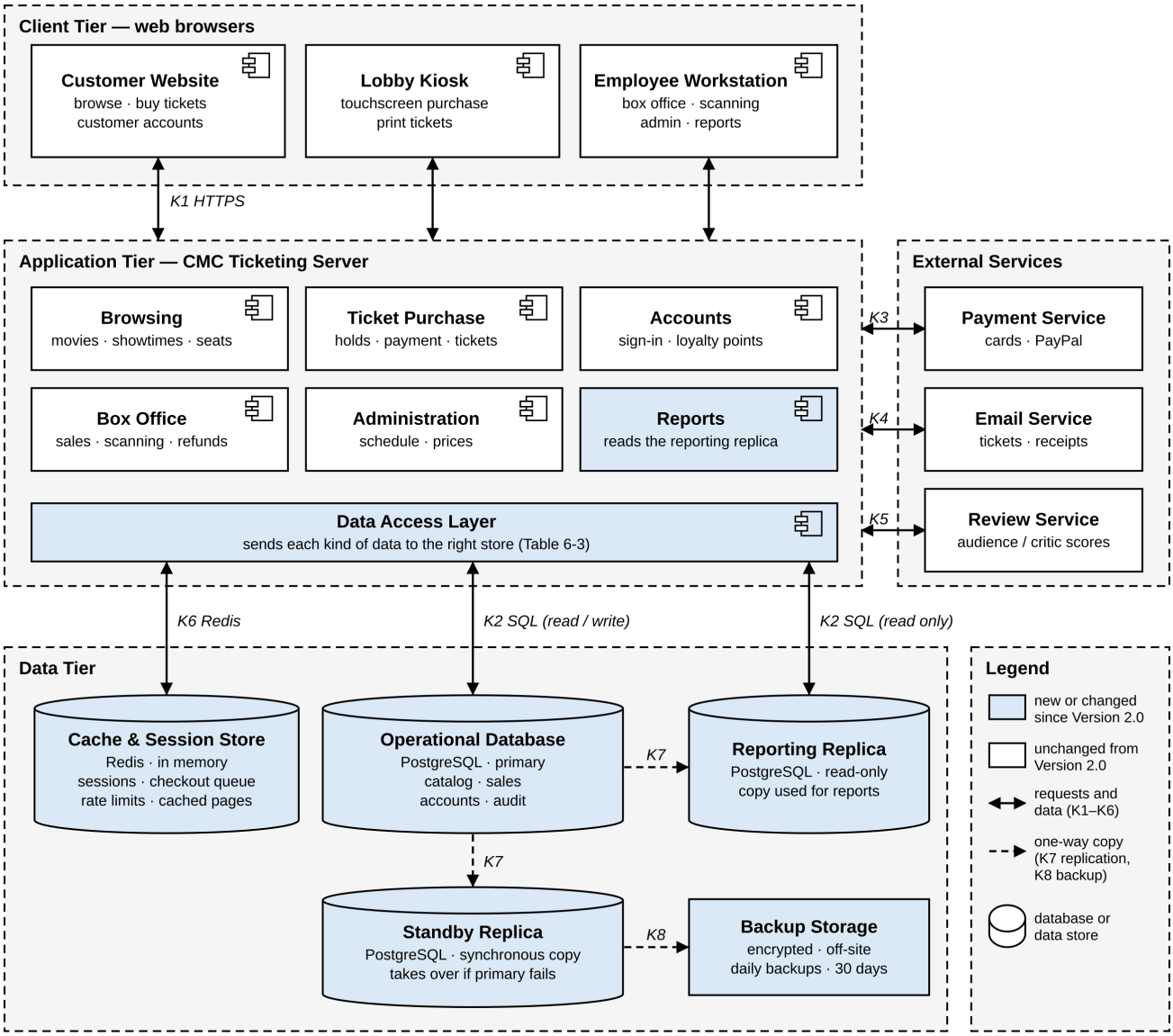


Figure 5. Updated software architecture of the CMC Ticketing System, showing the data tier

Components. Table 6-1 describes the new and changed components. All other components are as described in Table 4-1.

Table 6-1. *New and changed components*

Component	Tier	Description
Operational Database	Data	A PostgreSQL relational database. It is the system of record: the one place where every permanent piece of data (theaters, showtimes, prices, seats, holds, tickets, transactions, accounts, and audit records) is stored. Its tables are grouped into four schemas (Section 6.3.2). Replaces the "Central Database" of Figure 2.
Standby Replica	Data	An exact copy of the Operational Database, kept in a separate building or cloud zone. It is updated *synchronously*: a purchase is confirmed only after it has been saved on both copies. If the primary database fails, the standby takes over automatically (NFR-A-04, NFR-R-05).

Component	Tier	Description
Reporting Replica	Data	A read-only copy of the Operational Database, updated a few seconds behind it. Sales reports run here, so large report queries never slow down ticket sales.
Cache & Session Store	Data	Redis, an in-memory key-value store. It holds short-lived data that must be fast and can be rebuilt if lost: sign-in sessions, the checkout waiting queue, bot-detection counters, and cached browse results.
Backup Storage	Data	Encrypted storage at a separate location. It receives a full backup every night and a continuous copy of the database's change log, and it keeps them for 30 days (NFR-R-04).
Data Access Layer *(changed)*	Server	Still loads and saves the objects of Figure 3, but now also sends each kind of data to the right store (Table 6-3): permanent data to the Operational Database, report queries to the Reporting Replica, and temporary data to Redis. The modules above it do not need to know which store they are using.
Reports *(changed)*	Server module	Reads from the Reporting Replica instead of the main database (FR-41 to FR-43). It has no permission to write any data.

Connectors. K1 and K3 to K5 are unchanged from Table 4-2. Table 6-2 describes the changed connector K2 and the three new connectors.

Table 6-2. *New and changed connectors*

ID	Connects	Protocol	Purpose
K2 *(changed)*	Data Access Layer and the Operational Database or Reporting Replica	SQL over TLS	Reads and saves permanent data. Each purchase, refund, and seat change is saved as one database transaction (Section 6.3.3). Report queries go to the Reporting Replica and can only read.
K6 *(new)*	Data Access Layer and Cache & Session Store	Redis protocol over TLS	Reads and saves sessions, queue positions, rate-limit counters, and cached pages. Every value is given an expiry time.
K7 *(new)*	Operational Database to Standby Replica and Reporting Replica	PostgreSQL streaming replication	Copies every change. The copy to the standby is synchronous, so no confirmed purchase can be lost; the copy to the reporting replica is asynchronous, because reports can be a few seconds behind.
K8 *(new)*	Standby Replica to Backup Storage	Encrypted file transfer	Sends the nightly full backup and the continuous change log. Backups are taken from the standby so they do not add load to the primary database.

Table 6-3. *Where each kind of data is stored*

Data	Store	Why
Theaters, auditorium layouts, movies, showtimes, prices	Operational Database (catalog schema)	Permanent; changed only by administrators
Seat status, ticket holds, payment attempts, transactions, tickets	Operational Database (sales schema)	Involves money and seats; must never be lost or sold twice
Customer and employee accounts, saved payment tokens	Operational Database (accounts schema)	Permanent and sensitive
Audit records and sign-in history	Operational Database (audit schema)	Permanent and must not be changed
Sign-in sessions, waiting queue, checkout-attempt counters	Cache & Session Store	Short-lived and needs to be fast; if lost, users sign in again

Data	Store	Why
Cached movie and showtime lists and review scores	Cache & Session Store	A copy only; rebuilt from the database within 60 seconds
Sales report queries	Reporting Replica	Large, read-only queries kept away from ticket sales

Example (updated from Section 4.2). A customer buys two deluxe seats on the website:

1. The Browsing module looks for the theater's showtime list in the Cache & Session Store (K6). If it is not there, it reads the list from the Operational Database (K2) and caches it for 60 seconds. Seats remaining and the seat map are always read from the database itself, never from the cache, so every channel sees the same availability (NFR-P-04).
2. When the customer chooses seats, the Ticket Purchase module locks the two seat rows, checks that both are AVAILABLE, marks them HELD, and creates the ticket hold, all in one database transaction. If either seat is already taken, nothing changes and the customer is asked to choose again.
3. Before contacting the Payment Service, the module saves a *payment attempt* marked PENDING, with a unique key for this checkout. It then sends the payment, with the same key, over K3.
4. When the payment is approved, one database transaction marks the attempt APPROVED, saves the transaction and the two tickets, and changes the seats from HELD to SOLD. The purchase is confirmed to the customer only after this transaction has also been saved on the Standby Replica (K7).
5. The Email Service sends the tickets (K4). Because the kiosks and box offices read the same database, the two seats already show as sold on every channel.

If the server fails between steps 3 and 4, the PENDING payment attempt tells the system to ask the Payment Service for the result before trying again, so the customer is never charged twice (Section 3.2.3.5).

6.2 Modifications from the Initial Design (Version 2.0)

Version 3.0 (Section 4.6) made no changes to the architecture. Table 6-4 lists every change made in this version, compared with the Version 2.0 design in Figure 2, and the requirement or problem that led to it.

Table 6-4. *Design changes since Version 2.0*

#	Change	Version 2.0	Version 4.0	Reason
1	Database technology chosen	"Central Database"; no technology named	PostgreSQL relational database	The data is closely related and seat and payment updates need all-or-nothing transactions (Section 6.4.1).
2	Database organized into schemas	One undivided database	Four schemas: catalog, sales, accounts, and audit	Makes clear which module owns which data and lets each module be given only the permissions it needs (NFR-S-04).
3	Standby Replica added	One database server, so a single point of failure	Synchronous standby copy with automatic failover	NFR-A-04 (one server failure stops sales for at most 5 minutes) and NFR-R-05 (no confirmed purchase lost).
4	Reporting Replica added	Reports ran on the same database as ticket sales	Reports run on a read-only copy	A large report during a busy release could slow checkout (NFR-P-03).
5	Cache & Session Store added	Sessions, the waiting queue, and bot counters had no defined home	Redis stores them with automatic expiry	Fast, short-lived data for FR-27, FR-45, NFR-S-08, and NFR-P-02.

#	Change	Version 2.0	Version 4.0	Reason
6	Backup Storage added	NFR-R-04 (daily backups, kept 30 days) had no component	Encrypted, off-site backup storage	Meets NFR-R-04 and allows recovery from a failure that affects both database copies.
7	Data Access Layer routes data	Talked to one database	Sends each kind of data to the right store (Table 6-3)	Keeps the modules unaware of where data is stored.
8	Connectors added or changed	K1 to K5	K2 now uses TLS and can reach the replica; K6, K7, and K8 added	Needed by changes 3 to 6; TLS meets NFR-S-01.
9	Supporting tables added	Not described	showtime_seat, hold_item, ticket_price, payment_attempt, saved_payment_method, and sign_in_log (Figure 6)	Store lists that a class keeps in memory and data that has no class of its own; payment_attempt makes the no-double-charge rule (NFR-R-01) possible.
10	Ticket codes stored as a hash	Not described	The database stores only a one-way hash of each ticket's validation code	A stolen copy of the database cannot be used to print valid tickets (NFR-S-06).

Effect on the test plan. The test sets in Section 5 still apply, and Figure 4 is not redrawn because it marks the same server components. Three test sets gain a data-tier target: TS-I1 and TS-S1 test the row locking on the showtime_seat table that prevents double sales; TS-S2's single-server-failure test (NFR-A-04) also stops the primary database and checks that the standby takes over within 5 minutes with no confirmed purchase missing; and TS-S3 checks that the Reports module's database user cannot change any data. A restore from Backup Storage is tested before deployment in Task 8.

6.3 Data Management Strategy

6.3.1 Overview

The CMC Ticketing System uses **SQL for all permanent data and a small in-memory store for temporary data.** The rule for deciding where data goes is:

- If losing or corrupting the data would lose money, a seat, a ticket, an account, or a record, it is stored in the **Operational Database** (PostgreSQL).
- If the data can be rebuilt from the database, or losing it only means a user has to sign in again, it may be stored in the **Cache & Session Store** (Redis).

This gives the system **two databases**: one PostgreSQL database and one Redis store. The Standby Replica and the Reporting Replica are copies of the PostgreSQL database that hold the same data; they are not separate databases and nothing is ever written to them directly.

6.3.2 Logical Organization of the Data

Figure 6 shows the logical data model. The tables of the Operational Database are grouped into four **schemas**, or groups of tables inside one database. They keeps related tables together and makes permissions easy to manage, but tables in different schemas can still be joined in one query and updated in one transaction. The Redis keys are shown on the right. To keep the figure readable, relationships are drawn as lines inside each schema; a link to a table in another schema is written as "FK → table".

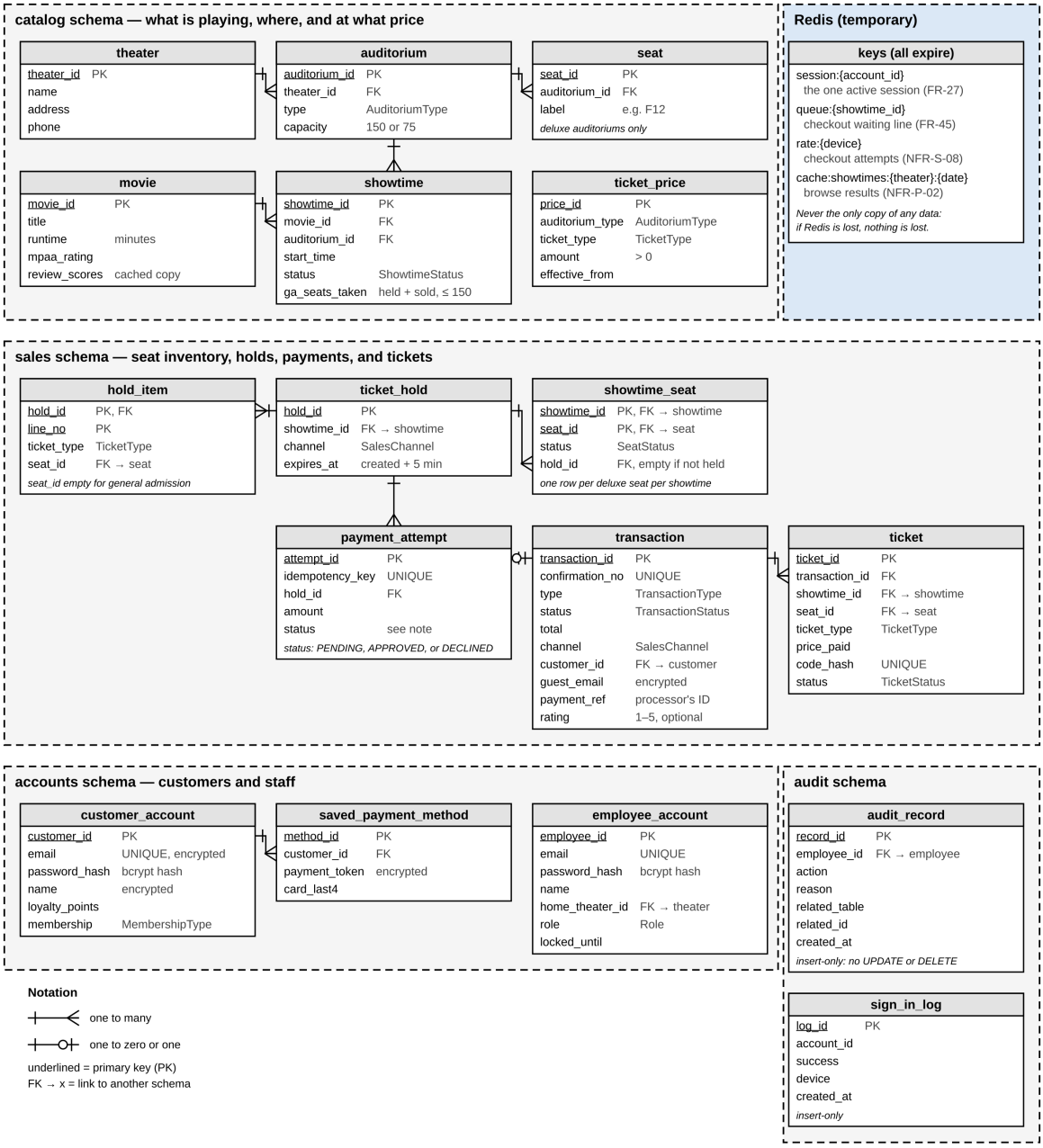


Figure 6. Logical data model of the CMC Ticketing System, grouped by schema

Table 6-5. *Schemas, their tables, and which modules use them*

Schema	Tables	Holds	Written by	Read by
catalog	theater, auditorium, seat, movie, showtime, ticket_price	What is playing, where, when, and at what price	Administration (Browsing refreshes cached review scores)	All modules
sales	showtime_seat, ticket_hold, hold_item, payment_attempt, transaction, ticket	Seat availability, holds, payments, and tickets	Ticket Purchase, Box Office, Administration (overrides and cancellations)	Accounts (purchase history), Reports (through the replica)

Schema	Tables	Holds	Written by	Read by
accounts	customer_account, saved_payment_method, employee_account	Customer and staff accounts	Accounts (Ticket Purchase adds loyalty points)	Ticket Purchase, Box Office, Administration (role checks)
audit	audit_record, sign_in_log	Permanent record of sensitive actions and sign-in attempts	Box Office, Administration, Accounts (insert only)	Administrators and managers

From classes to tables. Each class in Section 4.4 is stored in a table of the same name, with these differences:

- **Account** is abstract, so it has no table. Its sign-in fields are repeated in customer_account and employee_account (Section 6.4.3 explains why). Each account's single active session (FR-27) is kept in Redis, where signing in on a new device simply replaces the old session.
- **Lists become child tables.** Showtime's seat-status map becomes one showtime_seat row per seat; TicketHold's ticket types and seats become hold_item rows; a Transaction's tickets are ticket rows that point to it; and a CustomerAccount's saved payment tokens become saved_payment_method rows.
- **PaymentGateway** is an interface with no data, so it has no table.
- **Three tables have no class:** ticket_price (the prices administrators configure, FR-37), payment_attempt (each call to the Payment Service, NFR-R-01), and sign_in_log (sign-in attempts, NFR-S-05).

Regular and deluxe seating. A deluxe showtime has 75 showtime_seat rows, created when the showtime is scheduled, each AVAILABLE, HELD, or SOLD. A regular showtime has no seat rows; instead its ga_seats_taken counter counts held and sold general-admission tickets, and the database refuses any change that would take it past the auditorium's capacity of 150 (FR-10).

6.3.3 Data Integrity and Consistency

- **One system of record.** Every channel reads and writes the same Operational Database, so prices and availability are the same everywhere (SI-03). Seat availability is never cached.
- **Database rules as a safety net.** Primary keys, foreign keys, UNIQUE rules (confirmation numbers, ticket code hashes, payment-attempt keys, and account emails), and CHECK rules (prices above zero, ratings from 1 to 5, at most 20 items per hold, and general-admission seats within capacity) are enforced by the database itself. Even if the application code had a bug, the database would refuse the invalid data.
- **No seat sold twice (FR-12, NFR-R-02).** To hold seats, the Ticket Purchase module locks the requested showtime_seat rows inside a transaction. If two customers ask for the same seat at the same moment, the second request waits until the first finishes, then sees the seat as HELD and fails. A hold takes all of the requested seats or none of them.
- **Expired holds (FR-13, NFR-R-03).** Each hold stores its expiry time. Any hold past that time is treated as released immediately, and a cleanup task runs every 5 seconds to return the seats to AVAILABLE, well within the 10-second limit.
- **Exactly one charge (NFR-R-01).** The payment attempt is saved with a unique key before the Payment Service is called, and the same key is sent to the service. A repeated submission reuses the key, so it cannot create a second charge or a second transaction.
- **All-or-nothing purchases (FR-18, FR-19).** After the payment is approved, the transaction, its tickets, and the SOLD seat changes are saved in a single database transaction: either all of them are saved or none are.
- **Write-once audit records (FR-39).** The application's database users may add rows to the audit schema and read them, but they have no permission to change or delete them.

6.3.4 Sensitive Data and Security

The system stores personal and payment-related data, so each kind of data is protected according to how sensitive it is. Table 6-6 shows how.

Table 6-6. *Sensitive data and how it is protected*

Data	Examples	Sensitivity	Protection
Card details	Card number, security code	Highest	Never stored or seen by CMC. They are typed into the payment processor's own form, and CMC keeps only a token and the last four digits (NFR-S-03, PCI DSS).
Payment tokens	Saved payment methods, payment references	High	Encrypted in their own columns, with the key kept outside the database in a key-management service. Only the Ticket Purchase module can decrypt them.
Passwords	password_hash	High	Stored only as salted bcrypt hashes, which cannot be turned back into passwords (NFR-S-02).
Personal information	Names, account emails, guest emails	Medium	Encrypted at rest (NFR-S-01). A customer sees only their own data; employees see it only when looking up a transaction (FR-30).
Ticket validation codes	code_hash	Medium	Codes are 128-bit random values (NFR-S-06); only their hash is stored. A scanner's code is hashed and looked up, so a copy of the database cannot be used to make tickets.
Audit and sign-in records	audit_record, sign_in_log	Medium	Insert-only (Section 6.3.3); readable only by administrators and managers.
Sessions	session:{account_id} keys in Redis	Medium	Random session tokens that expire automatically; sent only over TLS.
Catalog	Movies, showtimes, prices	Low (public)	Anyone may read it; only administrators may change it, and each change is audited.

In addition, all data stores sit on a private network that only the CMC server can reach; the disks and backups are encrypted; and each server module connects with its own database user that has access only to the schemas listed for it in Table 6-5. Kiosks store no customer data at all (NFR-S-07).

6.3.5 Backup, Recovery, and Retention

- **Server failure.** If the primary database fails, the Standby Replica is promoted automatically, usually within one minute, and no confirmed purchase is lost because every purchase was already saved on the standby (NFR-A-04, NFR-R-05).
- **Backups.** A full backup is taken every night and the database's change log is copied to Backup Storage continuously. Together they allow the database to be restored to any moment in the last 30 days (NFR-R-04), for example to undo a mistake made by an administrator. A test restore is performed every month during the maintenance window (NFR-A-02).
- **Redis.** The Cache & Session Store is not backed up. If it is lost, caches refill from the database, customers sign in again, and customers waiting in a queue rejoin it.
- **Software updates.** Changes to the database structure are written as versioned scripts stored in the GitHub repository and applied during the maintenance window, so updates do not lose data (NFR-M-04).

Table 6-7. *How long data is kept (based on assumed CMC policies)*

Data	Kept for	Then
Transactions, tickets, and audit records	7 years (assumed CMC financial-records policy)	Moved to archive storage
Guest email addresses on transactions	1 year	Removed; the transaction stays for reports
Ticket holds and hold items	1 day after they expire or become a purchase	Deleted
Payment attempts and sign-in log	1 year	Deleted
Customer accounts	Until the customer closes the account	Personal data deleted; past transactions kept without the customer's name or email
Backups	30 days (NFR-R-04)	Deleted

6.4 Design Decisions, Alternatives, and Tradeoffs

6.4.1 SQL or NoSQL

The main technology decision was between a relational (SQL) database and a NoSQL database such as a document store (for example, MongoDB) or a key-value store (for example, Amazon DynamoDB). Table 6-8 compares them for this system.

Table 6-8. *Relational (SQL) compared with NoSQL for the CMC Ticketing System*

Consideration	Relational database (chosen)	NoSQL database
Shape of the data	Fits well: theaters, auditoriums, showtimes, seats, tickets, and transactions are separate things linked to each other, exactly as in Figure 3.	Fits less well: related data is either copied into several documents, which can then disagree, or joined together in application code.
Seats and money	Built-in transactions with locking make "no seat sold twice" and "tickets only after payment" simple and reliable.	Possible, but multi-record transactions are more limited or slower in many NoSQL systems, and more of the work falls on our code.
Reports	Sales by theater, movie, and date range are ordinary SQL queries (FR-42).	Each new report often needs its own copy of the data or extra processing.
Changing the structure	Needs a migration script whenever a column is added.	Flexible: new fields can be added at any time.
Scaling	One server, plus replicas for reading, is enough at CMC's size; growing far beyond that is harder.	Spreads easily across many servers, which matters at very large scale.

Decision. We chose a relational database because the system's most important requirements, never selling a seat twice and never charging twice, depend on reliable transactions, and because the data is naturally relational. NoSQL's main strengths, a flexible structure and very large scale, do not matter much here: CMC's data has a fixed, well-understood structure, and even if each of the 20 theaters ran about 60 showings a day (roughly 1,200 showtimes), the system would sell at most about 180,000 tickets a day, which one PostgreSQL server handles easily. We chose PostgreSQL over other relational databases because it is free, open source, widely used, and supported by every major cloud provider. The tradeoff we accept is that structural changes need migration scripts, and that growing to thousands of theaters would require a larger redesign.

6.4.2 How Many Databases

Table 6-9 compares four ways of dividing the data among databases.

Table 6-9. *Options for the number of databases*

Option	Benefits	Drawbacks
A. One database for everything (Version 2.0)	Simplest to build and run.	Sessions, queue counters, and caches add constant small writes to the main database; reports compete with checkout; one server is a single point of failure.
B. One relational database plus one in-memory store, with replicas (chosen)	All money and seat data is in one place, so it can be updated in one transaction; fast temporary data is kept apart; replicas give failover and a separate place for reports.	Two technologies to learn and run; Redis data can be lost (acceptable, because by design nothing in it is the only copy).
C. A separate database for each module (microservices)	Modules can be developed, scaled, and deployed on their own.	A purchase touches seats, tickets, transactions, and accounts, so one purchase would span several databases and need complex coordination to stay consistent. Too complex for three developers and 20 theaters.
D. A separate database at each theater	Box offices could keep selling if a theater lost its internet connection.	Breaks the shared, real-time inventory required by SI-03 and NFR-P-04, since the website would have to check 20 databases; company-wide reports would need to combine 20 databases.

Decision. We chose option B. It keeps the critical data together, which is the main reason for choosing SQL in the first place, while fixing the weaknesses of option A. Option D would only be worth reconsidering if the assumption in Section 2.5 that every theater has a reliable internet connection turned out to be false.

6.4.3 Other Design Decisions

Table 6-10. *Other data design decisions*

Decision	Our choice	Alternative considered	Tradeoff
Where to keep ticket holds	In PostgreSQL, as HELD seat rows with an expiry time	In Redis, which can delete a key automatically when its time runs out	Redis would be slightly faster and simpler for timing, but seat status would then live in two places that could disagree, which risks selling a seat twice. Correctness matters more than a few milliseconds.
Where reports run	On a read-only replica	(a) On the main database; (b) in a separate data warehouse loaded every night	(a) is simpler but lets reports slow down sales. (b) is better for long-term trend analysis, but its data would be up to a day old and it is another system to build. The replica is a few seconds behind, which is fine for sales reports. CMC could add a warehouse later.
Splitting data inside the database	Four schemas in one database	Separate databases for each area	Separate databases would isolate the areas more strongly but would lose single-transaction purchases across sales and accounts. Schemas give clear ownership and permissions without that cost.
Customer and employee accounts	Two separate tables	One account table with a type column, matching the abstract Account class	One table would mean less repeated code for sign-in, but staff accounts need different permissions and stricter protection. Separate tables let the database enforce that, at the cost of sign-in code checking two tables.

Decision	Our choice	Alternative considered	Tradeoff
Who runs the databases	A managed cloud database service (for example, Amazon RDS for PostgreSQL and a managed Redis service)	CMC's own servers	The managed service handles backups, failover, and security updates for a monthly fee, and ties CMC to one provider. Because PostgreSQL and Redis are open source, CMC could move to another provider or its own servers later without rewriting the system.